

SDK REFERENCES

Agent SDK reference - Python

Complete API reference for the Python Agent SDK, including all functions, types, and classes.

Installation

```
pip install claude-agent-sdk
```

Choosing between `query()` and `ClaudeSDKClient`

The Python SDK provides two ways to interact with Claude Code:

Quick comparison

Feature	<code>query()</code>	<code>ClaudeSDKClient</code>
Session	Creates a new session by default	Reuses same session
Conversation	Single exchange	Multiple exchanges in same context
Connection	Managed automatically	Manual control
Streaming Input	 Supported	 Supported
Interrupts	 Not supported	 Supported
Hooks	 Supported	 Supported
Custom Tools	 Supported	 Supported
Continue Chat	Manual via <code>continue_conversation</code> or <code>resume</code>	 Automatic
Use Case	One-off tasks	Continuous conversations

When to use `query()` (one-off tasks)

Best for:

- One-off questions where you don't need conversation history
- Independent tasks that don't require context from previous exchanges
- Simple automation scripts
- When you want a fresh start each time

When to use `ClaudeSDKClient` (continuous conversation)

Best for:

- **Continuing conversations** - When you need Claude to remember context
- **Follow-up questions** - Building on previous responses
- **Interactive applications** - Chat interfaces, REPLs
- **Response-driven logic** - When next action depends on Claude's response
- **Session control** - Managing conversation lifecycle explicitly

Functions

`query()`

Creates a new session for each interaction with Claude Code by default. Returns an async iterator that yields messages as they arrive. Each call to `query()` starts fresh with no memory of previous interactions unless you pass `continue_conversation=True` or `resume` in `ClaudeAgentOptions`.

See [Sessions](#).

```
async def query(  
    *,  
    prompt: str | AsyncIterable[dict[str, Any]],  
    options: ClaudeAgentOptions | None = None,  
    transport: Transport | None = None  
) -> AsyncIterator[Message]
```

Parameters

Parameter	Type	Description
prompt	str AsyncIterable[dict]	The input prompt as a string or async iterable for streaming mode
options	ClaudeAgentOptions None	Optional configuration object (defaults to ClaudeAgentOptions() if None)
transport	Transport None	Optional custom transport for communicating with the CLI process

Returns

Returns an AsyncIterator[Message] that yields messages from the conversation.

Example - With options

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main():
    options = ClaudeAgentOptions(
        system_prompt="You are an expert Python developer",
        permission_mode="acceptEdits",
        cwd="/home/user/project",
    )

    async for message in query(prompt="Create a Python web
server", options=options):
        print(message)

asyncio.run(main())
```

tool()

Decorator for defining MCP tools with type safety.

```
def tool(  
    name: str,  
    description: str,  
    input_schema: type | dict[str, Any],  
    annotations: ToolAnnotations | None = None  
) -> Callable[[Callable[[Any], Awaitable[dict[str, Any]]]],  
    SdkMcpTool[Any]]
```

Parameters

Parameter	Type	Description
name	str	Unique identifier for the tool
description	str	Human-readable description of what the tool does
input_schema	type dict[str, Any]	Schema defining the tool's input parameters (see below)
annotations	<u>ToolAnnotations</u> None	Optional MCP tool annotations providing behavioral hints to clients

Input schema options

- 1. **Simple type mapping** (recommended):

```
{"text": str, "count": int, "enabled": bool}
```

- 2. **JSON Schema format** (for complex validation):

```
{  
    "type": "object",  
    "properties": {  
        "text": {"type": "string"},  
        "count": {"type": "integer", "minimum": 0},  
    },  
    "required": ["text"],  
}
```

Returns

A decorator function that wraps the tool implementation and returns an `SdkMcpTool` instance.

Example

```
from claude_agent_sdk import tool
from typing import Any

@tool("greet", "Greet a user", {"name": str})
async def greet(args: dict[str, Any]) -> dict[str, Any]:
    return {"content": [{"type": "text", "text": f"Hello, {args['name']}!"}]}
```

ToolAnnotations

Re-exported from `mcp.types` (also available as `from claude_agent_sdk import ToolAnnotations`). All fields are optional hints; clients should not rely on them for security decisions.

Field	Type	Default	Description
<code>title</code>	<code>str None</code>	<code>None</code>	Human-readable title for the tool
<code>readOnlyHint</code>	<code>bool None</code>	<code>False</code>	If <code>True</code> , the tool does not modify its environment
<code>destructiveHint</code>	<code>bool None</code>	<code>True</code>	If <code>True</code> , the tool may perform destructive updates (only meaningful when <code>readOnlyHint</code> is <code>False</code>)
<code>idempotentHint</code>	<code>bool None</code>	<code>False</code>	If <code>True</code> , repeated calls with the same arguments have no additional effect (only meaningful when <code>readOnlyHint</code> is <code>False</code>)
<code>openWorldHint</code>	<code>bool None</code>	<code>True</code>	If <code>True</code> , the tool interacts with external entities (for example, web search). If <code>False</code> , the tool's domain is closed (for example, a memory tool)

```
from claude_agent_sdk import tool, ToolAnnotations
from typing import Any

@tool(
    "search",
    "Search the web",
    {"query": str},
    annotations=ToolAnnotations(readOnlyHint=True,
openWorldHint=True),
)
async def search(args: dict[str, Any]) -> dict[str, Any]:
    return {"content": [{"type": "text", "text": f"Results for:
{args['query']}"}]}
```

create_sdk_mcp_server()

Create an in-process MCP server that runs within your Python application.

```
def create_sdk_mcp_server(
    name: str,
    version: str = "1.0.0",
    tools: list[SdkMcpTool[Any]] | None = None
) -> McpSdkServerConfig
```

Parameters

Parameter	Type	Default	Description
name	str	-	Unique identifier for the server
version	str	"1.0.0"	Server version string
tools	list[SdkMcpTool[Any]] None	None	List of tool functions created with @tool decorator

Returns

Returns an McpSdkServerConfig object that can be passed to ClaudeAgentOptions.mcp_servers .

Example

```
from claude_agent_sdk import tool, create_sdk_mcp_server

@tool("add", "Add two numbers", {"a": float, "b": float})
async def add(args):
    return {"content": [{"type": "text", "text": f"Sum: {args['a'] + args['b']}"}]}

@tool("multiply", "Multiply two numbers", {"a": float, "b": float})
async def multiply(args):
    return {"content": [{"type": "text", "text": f"Product: {args['a'] * args['b']}"}]}

calculator = create_sdk_mcp_server(
    name="calculator",
    version="2.0.0",
    tools=[add, multiply], # Pass decorated functions
)

# Use with Claude
options = ClaudeAgentOptions(
    mcp_servers={"calc": calculator},
    allowed_tools=["mcp__calc__add", "mcp__calc__multiply"],
)
```

list_sessions()

Lists past sessions with metadata. Filter by project directory or list sessions across all projects. Synchronous; returns immediately.

```
def list_sessions(
    directory: str | None = None,
    limit: int | None = None,
    include_worktrees: bool = True
) -> list[SDKSessionInfo]
```

Parameters

Parameter	Type	Default	Description
directory	str None	None	Directory to list sessions for. When omitted, returns sessions across all projects
limit	int None	None	Maximum number of sessions to return
include_worktrees	bool	True	When <code>directory</code> is inside a git repository, include sessions from all worktree paths

Return type: SDKSessionInfo

Property	Type	Description
session_id	str	Unique session identifier
summary	str	Display title: custom title, auto-generated summary, or first prompt
last_modified	int	Last modified time in milliseconds since epoch
file_size	int None	Session file size in bytes (<code>None</code> for remote storage backends)
custom_title	str None	User-set session title
first_prompt	str None	First meaningful user prompt in the session
git_branch	str None	Git branch at the end of the session
cwd	str None	Working directory for the session
tag	str None	User-set session tag (see <code>tag_session()</code>)
created_at	int None	Session creation time in milliseconds since epoch

Example

Print the 10 most recent sessions for a project. Results are sorted by `last_modified` descending, so the first item is the newest. Omit `directory` to search across all projects.

```
from claude_agent_sdk import list_sessions

for session in list_sessions(directory="/path/to/project",
limit=10):
    print(f"{session.summary} ({session.session_id})")
```


get_session_messages()

Retrieves messages from a past session. Synchronous; returns immediately.

```
def get_session_messages(  
    session_id: str,  
    directory: str | None = None,  
    limit: int | None = None,  
    offset: int = 0  
) -> list[SessionMessage]
```

Parameters

Parameter	Type	Default	Description
session_id	str	required	The session ID to retrieve messages for
directory	str None	None	Project directory to look in. When omitted, searches all projects
limit	int None	None	Maximum number of messages to return
offset	int	0	Number of messages to skip from the start

Return type: SessionMessage

Property	Type	Description
type	Literal["user", "assistant"]	Message role
uuid	str	Unique message identifier
session_id	str	Session identifier
message	Any	Raw message content
parent_tool_use_id	None	Reserved for future use

Example

```
from claude_agent_sdk import list_sessions, get_session_messages

sessions = list_sessions(limit=1)
if sessions:
    messages = get_session_messages(sessions[0].session_id)
    for msg in messages:
        print(f"[{msg.type}] {msg.uuid}")
```

get_session_info()

Reads metadata for a single session by ID without scanning the full project directory. Synchronous; returns immediately.

```
def get_session_info(
    session_id: str,
    directory: str | None = None,
) -> SDKSessionInfo | None
```

Parameters

Parameter	Type	Default	Description
session_id	str	required	UUID of the session to look up
directory	str None	None	Project directory path. When omitted, searches all project directories

Returns `SDKSessionInfo`, or `None` if the session is not found.

Example

Look up a single session's metadata without scanning the project directory. Useful when you already have a session ID from a previous run.

```
from claude_agent_sdk import get_session_info

info = get_session_info("550e8400-e29b-41d4-a716-446655440000")
if info:
    print(f"{info.summary} (branch: {info.git_branch}, tag: {info.tag})")
```

rename_session()

Renames a session by appending a custom-title entry. Repeated calls are safe; the most recent title wins. Synchronous.

```
def rename_session(
    session_id: str,
    title: str,
    directory: str | None = None,
) -> None
```

Parameters

Parameter	Type	Default	Description
<code>session_id</code>	<code>str</code>	required	UUID of the session to rename
<code>title</code>	<code>str</code>	required	New title. Must be non-empty after stripping whitespace
<code>directory</code>	<code>str None</code>	<code>None</code>	Project directory path. When omitted, searches all project directories

Raises `ValueError` if `session_id` is not a valid UUID or `title` is empty; `FileNotFoundError` if the session cannot be found.

Example

Rename the most recent session so it's easier to find later. The new title appears in

`SDKSessionInfo.custom_title` on subsequent reads.

```
from claude_agent_sdk import list_sessions, rename_session

sessions = list_sessions(directory="/path/to/project", limit=1)
if sessions:
    rename_session(sessions[0].session_id, "Refactor auth
module")
```

tag_session()

Tags a session. Pass `None` to clear the tag. Repeated calls are safe; the most recent tag wins. Synchronous.

```
def tag_session(
    session_id: str,
    tag: str | None,
    directory: str | None = None,
) -> None
```

Parameters

Parameter	Type	Default	Description
<code>session_id</code>	<code>str</code>	required	UUID of the session to tag
<code>tag</code>	<code>str None</code>	required	Tag string, or <code>None</code> to clear. Unicode-sanitized before storing
<code>directory</code>	<code>str None</code>	<code>None</code>	Project directory path. When omitted, searches all project directories

Raises `ValueError` if `session_id` is not a valid UUID or `tag` is empty after sanitization; `FileNotFoundError` if the session cannot be found.

Example

Tag a session, then filter by that tag on a later read. Pass `None` to clear an existing tag.

```
from claude_agent_sdk import list_sessions, tag_session

# Tag a session
tag_session("550e8400-e29b-41d4-a716-446655440000", "needs-
review")

# Later: find all sessions with that tag
for session in list_sessions(directory="/path/to/project"):
    if session.tag == "needs-review":
        print(session.summary)
```

Classes

ClaudeSDKClient

Maintains a conversation session across multiple exchanges. This is the Python equivalent of how the TypeScript SDK's `query()` function works internally - it creates a client object that can continue conversations.

Key Features

- **Session continuity:** Maintains conversation context across multiple `query()` calls
- **Same conversation:** The session retains previous messages
- **Interrupt support:** Can stop execution mid-task
- **Explicit lifecycle:** You control when the session starts and ends
- **Response-driven flow:** Can react to responses and send follow-ups
- **Custom tools and hooks:** Supports custom tools (created with `@tool` decorator) and hooks

```

class ClaudeSDKClient:
    def __init__(self, options: ClaudeAgentOptions | None = None,
transport: Transport | None = None)
        async def connect(self, prompt: str | AsyncIterable[dict] |
None = None) -> None
        async def query(self, prompt: str | AsyncIterable[dict],
session_id: str = "default") -> None
        async def receive_messages(self) -> AsyncIterator[Message]
        async def receive_response(self) -> AsyncIterator[Message]
        async def interrupt(self) -> None
        async def set_permission_mode(self, mode: str) -> None
        async def set_model(self, model: str | None = None) -> None
        async def rewind_files(self, user_message_id: str) -> None
        async def get_mcp_status(self) -> McpStatusResponse
        async def reconnect_mcp_server(self, server_name: str) ->
None
        async def toggle_mcp_server(self, server_name: str, enabled:
bool) -> None
        async def stop_task(self, task_id: str) -> None
        async def get_server_info(self) -> dict[str, Any] | None
        async def disconnect(self) -> None

```

Methods

Method	Description
<code>__init__(options)</code>	Initialize the client with optional configuration
<code>connect(prompt)</code>	Connect to Claude with an optional initial prompt or message stream
<code>query(prompt, session_id)</code>	Send a new request in streaming mode
<code>receive_messages()</code>	Receive all messages from Claude as an async iterator
<code>receive_response()</code>	Receive messages until and including a ResultMessage
<code>interrupt()</code>	Send interrupt signal (only works in streaming mode)
<code>set_permission_mode(mode)</code>	Change the permission mode for the current session
<code>set_model(model)</code>	Change the model for the current session. Pass <code>None</code> to reset to default
<code>rewind_files(user_message_id)</code>	Restore files to their state at the specified user message. Requires <code>enable_file_checkpointing=True</code> . See File checkpointing
<code>get_mcp_status()</code>	Get the status of all configured MCP servers. Returns <code>McpStatusResponse</code>
<code>reconnect_mcp_server(server_na</code>	Retry connecting to an MCP server that failed or was disconnected

Method	Description
<code>me)</code>	
<code>toggle_mcp_server(server_name, enabled)</code>	Enable or disable an MCP server mid-session. Disabling removes its tools
<code>stop_task(task_id)</code>	Stop a running background task. A <code>TaskNotificationMessage</code> with status <code>"stopped"</code> follows in the message stream
<code>get_server_info()</code>	Get server information including session ID and capabilities
<code>disconnect()</code>	Disconnect from Claude

Context Manager Support

The client can be used as an async context manager for automatic connection management:

```
async with ClaudeSDKClient() as client:
    await client.query("Hello Claude")
    async for message in client.receive_response():
        print(message)
```

Important: When iterating over messages, avoid using `break` to exit early as this can cause asyncio cleanup issues. Instead, let the iteration complete naturally or use flags to track when you've found what you need.

Example - Continuing a conversation

```
import asyncio
from claude_agent_sdk import ClaudeSDKClient, AssistantMessage,
TextBlock, ResultMessage

async def main():
    async with ClaudeSDKClient() as client:
        # First question
        await client.query("What's the capital of France?")

        # Process response
        async for message in client.receive_response():
            if isinstance(message, AssistantMessage):
                for block in message.content:
                    if isinstance(block, TextBlock):
                        print(f"Claude: {block.text}")

        # Follow-up question - the session retains the previous
context
        await client.query("What's the population of that city?")

        async for message in client.receive_response():
            if isinstance(message, AssistantMessage):
                for block in message.content:
                    if isinstance(block, TextBlock):
                        print(f"Claude: {block.text}")

        # Another follow-up - still in the same conversation
        await client.query("What are some famous landmarks
there?")

        async for message in client.receive_response():
            if isinstance(message, AssistantMessage):
                for block in message.content:
                    if isinstance(block, TextBlock):
                        print(f"Claude: {block.text}")

asyncio.run(main())
```


Example - Streaming input with ClaudeSDKClient

```
import asyncio
from claude_agent_sdk import ClaudeSDKClient

async def message_stream():
    """Generate messages dynamically."""
    yield {
        "type": "user",
        "message": {"role": "user", "content": "Analyze the
following data:"},
    }
    await asyncio.sleep(0.5)
    yield {
        "type": "user",
        "message": {"role": "user", "content": "Temperature:
25°C, Humidity: 60%"},
    }
    await asyncio.sleep(0.5)
    yield {
        "type": "user",
        "message": {"role": "user", "content": "What patterns do
you see?"},
    }

async def main():
    async with ClaudeSDKClient() as client:
        # Stream input to Claude
        await client.query(message_stream())

        # Process response
        async for message in client.receive_response():
            print(message)

        # Follow-up in same session
        await client.query("Should we be concerned about these
readings?")

        async for message in client.receive_response():
            print(message)
```

```
asyncio.run(main())
```

Example - Using interrupts

```
import asyncio
from claude_agent_sdk import ClaudeSDKClient, ClaudeAgentOptions,
ResultMessage

async def interruptible_task():
    options = ClaudeAgentOptions(allowed_tools=["Bash"],
permission_mode="acceptEdits")

    async with ClaudeSDKClient(options=options) as client:
        # Start a long-running task
        await client.query("Count from 1 to 100 slowly, using the
bash sleep command")

        # Let it run for a bit
        await asyncio.sleep(2)

        # Interrupt the task
        await client.interrupt()
        print("Task interrupted!")

        # Drain the interrupted task's messages (including its
ResultMessage)
        async for message in client.receive_response():
            if isinstance(message, ResultMessage):
                print(f"Interrupted task finished with subtype=
{message.subtype!r}")
                # subtype is "error_during_execution" for
interrupted tasks

        # Send a new command
        await client.query("Just say hello instead")

        # Now receive the new response
        async for message in client.receive_response():
            if isinstance(message, ResultMessage) and
message.subtype == "success":
                print(f"New result: {message.result}")

asyncio.run(interruptible_task())
```

❗ **Buffer behavior after interrupt:** `interrupt()` sends a stop signal but does not clear the message buffer. Messages already produced by the interrupted task, including its `ResultMessage` (with `subtype="error_during_execution"`), remain in the stream. You must drain them with `receive_response()` before reading the response to a new query. If you send a new query immediately after `interrupt()` and call `receive_response()` only once, you'll receive the interrupted task's messages, not the new query's response.

Example - Advanced permission control

```
from claude_agent_sdk import ClaudeSDKClient, ClaudeAgentOptions
from claude_agent_sdk.types import (
    PermissionResultAllow,
    PermissionResultDeny,
    ToolPermissionContext,
)

async def custom_permission_handler(
    tool_name: str, input_data: dict, context:
ToolPermissionContext
) -> PermissionResultAllow | PermissionResultDeny:
    """Custom logic for tool permissions."""

    # Block writes to system directories
    if tool_name == "Write" and input_data.get("file_path",
    "").startswith("/system/"):
        return PermissionResultDeny(
            message="System directory write not allowed",
            interrupt=True
        )

    # Redirect sensitive file operations
    if tool_name in ["Write", "Edit"] and "config" in
input_data.get("file_path", ""):
        safe_path = f"./sandbox/{input_data['file_path']}"
        return PermissionResultAllow(
            updated_input={**input_data, "file_path": safe_path}
        )

    # Allow everything else
    return PermissionResultAllow(updated_input=input_data)

async def main():
    options = ClaudeAgentOptions(
        can_use_tool=custom_permission_handler, allowed_tools=
["Read", "Write", "Edit"]
    )

    async with ClaudeSDKClient(options=options) as client:
        await client.query("Update the system config file")
```

```
async for message in client.receive_response():
    # Will use sandbox path instead
    print(message)
```

```
asyncio.run(main())
```

Types

❗ **@dataclass vs TypedDict** : This SDK uses two kinds of types. Classes decorated with `@dataclass` (such as `ResultMessage`, `AgentDefinition`, `TextBlock`) are object instances at runtime and support attribute access: `msg.result`. Classes defined with `TypedDict` (such as `ThinkingConfigEnabled`, `McpStdioServerConfig`, `SyncHookJSONOutput`) are **plain dicts at runtime** and require key access: `config["budget_tokens"]`, not `config.budget_tokens`. The `ClassName(field=value)` call syntax works for both, but only dataclasses produce objects with attributes.

SdkMcpTool

Definition for an SDK MCP tool created with the `@tool` decorator.

```
@dataclass
class SdkMcpTool(Generic[T]):
    name: str
    description: str
    input_schema: type[T] | dict[str, Any]
    handler: Callable[[T], Awaitable[dict[str, Any]]]
    annotations: ToolAnnotations | None = None
```

Property	Type	Description
name	str	Unique identifier for the tool
description	str	Human-readable description
input_schema	type[T] dict[str, Any]	Schema for input validation
handler	Callable[[T], Awaitable[dict[str, Any]]]	Async function that handles tool execution

Property	Type	Description
annotations	ToolAnnotations None	Optional MCP tool annotations (e.g., readOnlyHint , destructiveHint , openWorldHint).From mcp.types

Transport

Abstract base class for custom transport implementations. Use this to communicate with the Claude process over a custom channel (for example, a remote connection instead of a local subprocess).

 This is a low-level internal API. The interface may change in future releases. Custom implementations must be updated to match any interface changes.

```
from abc import ABC, abstractmethod
from collections.abc import AsyncIterator
from typing import Any

class Transport(ABC):
    @abstractmethod
    async def connect(self) -> None: ...

    @abstractmethod
    async def write(self, data: str) -> None: ...

    @abstractmethod
    def read_messages(self) -> AsyncIterator[dict[str, Any]]: ...

    @abstractmethod
    async def close(self) -> None: ...

    @abstractmethod
    def is_ready(self) -> bool: ...

    @abstractmethod
    async def end_input(self) -> None: ...
```

Method	Description
connect()	Connect the transport and prepare for communication

Method	Description
<code>write(data)</code>	Write raw data (JSON + newline) to the transport
<code>read_messages()</code>	Async iterator that yields parsed JSON messages
<code>close()</code>	Close the connection and clean up resources
<code>is_ready()</code>	Returns <code>True</code> if the transport can send and receive
<code>end_input()</code>	Close the input stream (for example, close stdin for subprocess transports)

```
Import: from claude_agent_sdk import Transport
```

ClaudeAgentOptions

Configuration dataclass for Claude Code queries.


```
@dataclass
```

```
class ClaudeAgentOptions:
```

```
    tools: list[str] | ToolsPreset | None = None
```

```
    allowed_tools: list[str] = field(default_factory=list)
```

```
    system_prompt: str | SystemPromptPreset | None = None
```

```
    mcp_servers: dict[str, McpServerConfig] | str | Path =  
field(default_factory=dict)
```

```
    strict_mcp_config: bool = False
```

```
    permission_mode: PermissionMode | None = None
```

```
    continue_conversation: bool = False
```

```
    resume: str | None = None
```

```
    max_turns: int | None = None
```

```
    max_budget_usd: float | None = None
```

```
    disallowed_tools: list[str] = field(default_factory=list)
```

```
    model: str | None = None
```

```
    fallback_model: str | None = None
```

```
    betas: list[SdkBeta] = field(default_factory=list)
```

```
    output_format: dict[str, Any] | None = None
```

```
    permission_prompt_tool_name: str | None = None
```

```
    cwd: str | Path | None = None
```

```
    cli_path: str | Path | None = None
```

```
    settings: str | None = None
```

```
    add_dirs: list[str | Path] = field(default_factory=list)
```

```
    env: dict[str, str] = field(default_factory=dict)
```

```
    extra_args: dict[str, str | None] =  
field(default_factory=dict)
```

```
    max_buffer_size: int | None = None
```

```
    debug_stderr: Any = sys.stderr # Deprecated
```

```
    stderr: Callable[[str], None] | None = None
```

```
    can_use_tool: CanUseTool | None = None
```

```
    hooks: dict[HookEvent, list[HookMatcher]] | None = None
```

```
    user: str | None = None
```

```
    include_partial_messages: bool = False
```

```
    include_hook_events: bool = False
```

```
    fork_session: bool = False
```

```
    agents: dict[str, AgentDefinition] | None = None
```

```
    setting_sources: list[SettingSource] | None = None
```

```
    sandbox: SandboxSettings | None = None
```

```
    plugins: list[SdkPluginConfig] = field(default_factory=list)
```

```
    max_thinking_tokens: int | None = None # Deprecated: use
```

```
thinking instead
```

```
    thinking: ThinkingConfig | None = None
```

```
    effort: EffortLevel | None = None
```

```
    enable_file_checkpointing: bool = False
```

```
session_store: SessionStore | None = None
session_store_flush: SessionStoreFlushMode = "batched"
```

Property	Type	Default	Description
tools	list[str] ToolsPreset None	None	Tools configuration. Use {"type": "preset", "preset": "claude_code"} for Claude Code's default tools
allowed_tools	list[str]	[]	Tools to auto-approve without prompting. This does not restrict Claude to only these tools; unlisted tools fall through to permission_mode and can_use_tool . Use disallowed_tools to block tools. See Permissions
system_prompt	str SystemPromptPreset None	None	System prompt configuration. Pass a string for custom prompt, or use {"type": "preset", "preset": "claude_code"} for Claude Code's system prompt. Add "append" to extend the preset
mcp_servers	dict[str, McpServerConfig] str Path	{}	MCP server configurations or path to config file
strict_mcp_config	bool	False	When True , use only the servers passed in mcp_servers and ignore project .mcp.json , user settings, plugin-provided MCP servers, and claude.ai connectors . Maps to the CLI <code>-strict-mcp-config</code> flag
permission_mode	PermissionMode None	None	Permission mode for tool usage
continue_conversation	bool	False	Continue the most recent conversation
resume	str None	None	Session ID to resume
max_turns	int None	None	Maximum agentic turns (tool-use round trips)
max_budget_usd	float None	None	Stop the query when the client-side cost estimate reaches this USD value. Compared against the same estimate as total_cost_usd ; see Track cost and usage for accuracy caveats
disallowed_tools	list[str]	[]	Tools to deny. A bare name such as "Bash" removes the tool from Claude's

Property	Type	Default	Description
			context. A scoped rule such as <code>"Bash(rm *)"</code> leaves the tool available and denies matching calls in every permission mode, including <code>bypassPermissions</code> . See Permissions
<code>enable_file_checkpo inting</code>	<code>bool</code>	<code>False</code>	Enable file change tracking for rewinding. See File checkpointing
<code>model</code>	<code>str None</code>	<code>None</code>	Claude model alias or full model name. See accepted values and provider-specific IDs
<code>fallback_model</code>	<code>str None</code>	<code>None</code>	Fallback model to use if the primary model fails
<code>betas</code>	<code>list[SdkBeta]</code>	<code>[]</code>	Beta features to enable. See SdkBeta for available options
<code>output_format</code>	<code>dict[str, Any] None</code>	<code>None</code>	Output format for structured responses (e.g., <code>{"type": "json_schema", "schema": {...}}</code>). See Structured outputs for details
<code>permission_prompt_t ool_name</code>	<code>str None</code>	<code>None</code>	MCP tool name for permission prompts
<code>cwd</code>	<code>str Path None</code>	<code>None</code>	Current working directory
<code>cli_path</code>	<code>str Path None</code>	<code>None</code>	Custom path to the Claude Code CLI executable
<code>settings</code>	<code>str None</code>	<code>None</code>	Path to settings file
<code>add_dirs</code>	<code>list[str Path]</code>	<code>[]</code>	Additional directories Claude can access
<code>env</code>	<code>dict[str, str]</code>	<code>{}</code>	Environment variables merged on top of the inherited process environment. See Environment variables for variables the underlying CLI reads, and Handle slow or stalled API responses for timeout-related variables
<code>extra_args</code>	<code>dict[str, str None]</code>	<code>{}</code>	Additional CLI arguments to pass directly to the CLI
<code>max_buffer_size</code>	<code>int None</code>	<code>None</code>	Maximum bytes when buffering CLI stdout
<code>debug_stderr</code>	<code>Any</code>	<code>sys.stderr</code>	<i>Deprecated</i> - File-like object for debug output. Use <code>stderr</code> callback instead
<code>stderr</code>	<code>Callable[[str], None] None</code>	<code>None</code>	Callback function for stderr output from CLI

Property	Type	Default	Description
can_use_tool	<code>CanUseTool</code> <code>None</code>	<code>None</code>	Tool permission callback function. See Permission types for details
hooks	<code>dict[HookEvent, list[HookMatcher]]</code> <code>None</code>	<code>None</code>	Hook configurations for intercepting events
user	<code>str</code> <code>None</code>	<code>None</code>	User identifier
include_partial_messages	<code>bool</code>	<code>False</code>	Include partial message streaming events. When enabled, StreamEvent messages are yielded
include_hook_events	<code>bool</code>	<code>False</code>	Include hook lifecycle events in the message stream as <code>HookEventMessage</code> objects
fork_session	<code>bool</code>	<code>False</code>	When resuming with <code>resume</code> , fork to a new session ID instead of continuing the original session
agents	<code>dict[str, AgentDefinition]</code> <code>None</code>	<code>None</code>	Programmatically defined subagents
plugins	<code>list[SdkPluginConfig]</code>	<code>[]</code>	Load custom plugins from local paths. See Plugins for details
sandbox	SandboxSettings <code>None</code>	<code>None</code>	Configure sandbox behavior programmatically. See Sandbox settings for details
setting_sources	<code>list[SettingSource]</code> <code>None</code>	<code>None</code> (CLI defaults: all sources)	Control which filesystem settings to load. Pass <code>[]</code> to disable user, project, and local settings. Endpoint-managed policy loads regardless; server-managed settings are fetched when the session authenticates with an organization credential on an eligible configuration . See Use Claude Code features
skills	<code>list[str]</code> <code>Literal["all"]</code> <code>None</code>	<code>None</code>	Skills available to the session. Pass <code>"all"</code> to enable every discovered skill, or a list of skill names. When set, the SDK adds the Skill tool to <code>allowed_tools</code> automatically. If you also pass <code>tools</code> , include <code>"Skill"</code> in that list. See Skills
max_thinking_tokens	<code>int</code> <code>None</code>	<code>None</code>	<i>Deprecated</i> - Maximum tokens for thinking blocks. Use <code>thinking</code> instead

Property	Type	Default	Description
thinking	ThinkingConfig None	None	Controls extended thinking behavior. Takes precedence over max_thinking_tokens
effort	EffortLevel None	None	Effort level for thinking depth. See adjust the effort level
session_store	SessionStore None	None	Mirror session transcripts to an external backend so any host can resume them. See Persist sessions to external storage
session_store_flush	Literal["batched", "eager"]	"batched"	When to flush mirrored transcript entries to session_store. "batched" flushes once per turn or when the buffer fills; "eager" triggers a background flush after every frame. Ignored when session_store is None

Handle slow or stalled API responses

The CLI subprocess reads several environment variables that control API timeouts and stall detection. Pass them through ClaudeAgentOptions.env :

```
options = ClaudeAgentOptions(  
    env={  
        "API_TIMEOUT_MS": "120000",  
        "CLAUDE_CODE_MAX_RETRIES": "2",  
        "CLAUDE_ASYNC_AGENT_STALL_TIMEOUT_MS": "120000",  
    },  
)
```

- API_TIMEOUT_MS : per-request timeout on the Anthropic client, in milliseconds. Default 600000 . Applies to the main loop and all subagents.
- CLAUDE_CODE_MAX_RETRIES : maximum API retries. Default 10 , capped at 15 . Each retry gets its own API_TIMEOUT_MS window, so worst-case wall time is roughly API_TIMEOUT_MS × (CLAUDE_CODE_MAX_RETRIES + 1) plus backoff. For unattended runs that need to wait through longer outages, set CLAUDE_CODE_RETRY_WATCHDOG=1 to retry capacity errors indefinitely.
- CLAUDE_ASYNC_AGENT_STALL_TIMEOUT_MS : stall watchdog for subagents launched with run_in_background . Default 600000 . Resets on each stream event; on stall it aborts the subagent, marks the task failed, and surfaces the error to the parent with any partial result. Does not apply to synchronous subagents.

- `CLAUDE_ENABLE_STREAM_WATCHDOG=1` with `CLAUDE_STREAM_IDLE_TIMEOUT_MS` : aborts the request when headers have arrived but the response body stops streaming. When `CLAUDE_ENABLE_STREAM_WATCHDOG` is unset, the default is server-controlled on the direct Anthropic API and off on other providers. `CLAUDE_STREAM_IDLE_TIMEOUT_MS` defaults to `300000` and is clamped to that minimum. The aborted request goes through the normal retry path.

OutputFormat

Configuration for structured output validation. Pass this as a `dict` to the `output_format` field on `ClaudeAgentOptions` :

```
# Expected dict shape for output_format
{
    "type": "json_schema",
    "schema": {...}, # Your JSON Schema definition
}
```

Field	Required	Description
type	Yes	Must be "json_schema" for JSON Schema validation
schema	Yes	JSON Schema definition for output validation

SystemPromptPreset

Configuration for using Claude Code's preset system prompt with optional additions.

```
class SystemPromptPreset(TypedDict):
    type: Literal["preset"]
    preset: Literal["claude_code"]
    append: NotRequired[str]
    exclude_dynamic_sections: NotRequired[bool]
```

Field	Required	Description
type	Yes	Must be "preset" to use a preset system prompt
preset	Yes	Must be "claude_code" to use Claude Code's system prompt

Field	Required	Description
append	No	Additional instructions to append to the preset system prompt
exclude_dynamic_sections	No	Move per-session context such as working directory, the git-repo flag, and auto-memory paths from the system prompt into the first user message. Improves prompt-cache reuse across users and machines. See Modify system prompts

SettingSource

Controls which filesystem-based configuration sources the SDK loads settings from.

```
SettingSource = Literal["user", "project", "local"]
```

Value	Description	Location
"user"	Global user settings	~/.claude/settings.json
"project"	Shared project settings (version controlled)	.claude/settings.json
"local"	Local project settings (not version controlled)	.claude/settings.local.json

Default behavior

When `setting_sources` is omitted or `None`, `query()` loads the same filesystem settings as the Claude Code CLI: user, project, and local. Endpoint-managed policy is loaded in all cases; server-managed settings are fetched when the session authenticates with an organization credential on an [eligible configuration](#). See [What settingSources does not control](#) for inputs that are read regardless of this option, and how to disable them.

Why use setting_sources

Disable filesystem settings:

```
# Do not load user, project, or local settings from disk
from claude_agent_sdk import query, ClaudeAgentOptions

async for message in query(
    prompt="Analyze this code",
    options=ClaudeAgentOptions(
        setting_sources=[]
    ),
):
    print(message)
```

❗ In Python SDK 0.1.59 and earlier, an empty list was treated the same as omitting the option, so `setting_sources=[]` did not disable filesystem settings. Upgrade to a newer release if you need an empty list to take effect. The TypeScript SDK is not affected.

Load all filesystem settings explicitly:

```
from claude_agent_sdk import query, ClaudeAgentOptions

async for message in query(
    prompt="Analyze this code",
    options=ClaudeAgentOptions(
        setting_sources=["user", "project", "local"]
    ),
):
    print(message)
```

Load only specific setting sources:

```
# Load only project settings, ignore user and local
async for message in query(
    prompt="Run CI checks",
    options=ClaudeAgentOptions(
        setting_sources=["project"] # Only .claude/settings.json
    ),
):
    print(message)
```

Testing and CI environments:


```
# Ensure consistent behavior in CI by excluding local settings
async for message in query(
    prompt="Run tests",
    options=ClaudeAgentOptions(
        setting_sources=["project"], # Only team-shared settings
        permission_mode="bypassPermissions",
    ),
):
    print(message)
```

SDK-only applications:

```
# Define everything programmatically.
# Pass [] to opt out of filesystem setting sources.
async for message in query(
    prompt="Review this PR",
    options=ClaudeAgentOptions(
        setting_sources=[],
        agents={...},
        mcp_servers={...},
        allowed_tools=["Read", "Grep", "Glob"],
    ),
):
    print(message)
```

Loading CLAUDE.md project instructions:

```
# Load project settings to include CLAUDE.md files
async for message in query(
    prompt="Add a new feature following project conventions",
    options=ClaudeAgentOptions(
        system_prompt={
            "type": "preset",
            "preset": "claude_code", # Use Claude Code's system
prompt
        },
        setting_sources=["project"], # Loads CLAUDE.md from
project
        allowed_tools=["Read", "Write", "Edit"],
    ),
):
    print(message)
```

Settings precedence

When multiple sources are loaded, settings are merged with this precedence (highest to lowest):

1. Local settings (`.claude/settings.local.json`)
2. Project settings (`.claude/settings.json`)
3. User settings (`~/.claude/settings.json`)

Programmatic options such as `agents` and `allowed_tools` override user, project, and local filesystem settings. Managed policy settings take precedence over programmatic options.

AgentDefinition

Configuration for a subagent defined programmatically.

```
@dataclass
class AgentDefinition:
    description: str
    prompt: str
    tools: list[str] | None = None
    disallowedTools: list[str] | None = None
    model: str | None = None
    skills: list[str] | None = None
    memory: Literal["user", "project", "local"] | None = None
    mcpServers: list[str | dict[str, Any]] | None = None
    initialPrompt: str | None = None
    maxTurns: int | None = None
    background: bool | None = None
    effort: EffortLevel | int | None = None
    permissionMode: PermissionMode | None = None
```

Field	Required	Description
description	Yes	Natural language description of when to use this agent
prompt	Yes	The agent's system prompt
tools	No	Array of allowed tool names. If omitted, inherits all tools
disallowedTools	No	Array of tool names to remove from the agent's tool set. MCP server-level patterns are also accepted: <code>mcp__server</code> or <code>mcp__server__*</code> removes every tool from that server, and <code>mcp__*</code> removes every MCP tool from any server
model	No	Model override for this agent. Accepts an alias such as <code>"sonnet"</code> , <code>"opus"</code> , <code>"haiku"</code> , or <code>"inherit"</code> , or a full model ID. If omitted, uses the main model
skills	No	List of skill names to preload into the agent's context at startup. Unlisted skills remain invocable through the Skill tool
memory	No	Memory source for this agent: <code>"user"</code> , <code>"project"</code> , or <code>"local"</code>
mcpServers	No	MCP servers available to this agent. Each entry is a server name or an inline <code>{name: config}</code> dict
initialPrompt	No	Auto-submitted as the first user turn when this agent runs as the main thread agent
maxTurns	No	Maximum number of agentic turns before the agent stops
background	No	Run this agent as a non-blocking background task when invoked

Field	Required	Description
effort	No	Reasoning effort level for this agent. Accepts a named level or an integer. See EffortLevel
permissionMode	No	Permission mode for tool execution within this agent. See PermissionMode

ⓘ `AgentDefinition` field names use camelCase, such as `disallowedTools`, `permissionMode`, and `maxTurns`. These names map directly to the wire format shared with the TypeScript SDK. This differs from `ClaudeAgentOptions`, which uses Python snake_case for the equivalent top-level fields such as `disallowed_tools` and `permission_mode`. Because `AgentDefinition` is a dataclass, passing a `snake_case` keyword raises a `TypeError` at construction time.

PermissionMode

Permission modes for controlling tool execution.

```
PermissionMode = Literal[
    "default", # Standard permission behavior
    "acceptEdits", # Auto-accept file edits
    "plan", # Planning mode - explore without editing
    "dontAsk", # Deny anything not pre-approved instead of
prompting
    "bypassPermissions", # Bypass permission checks; explicit
ask rules still prompt (use with caution)
]
```

EffortLevel

Effort levels for guiding thinking depth.

```
EffortLevel = Literal[
    "low", # Minimal thinking, fastest responses
    "medium", # Moderate thinking
    "high", # Deep reasoning
    "xhigh", # Extended reasoning (Opus 4.8 and Opus 4.7; falls
back to "high" on other models)
    "max", # Maximum effort
]
```

CanUseTool

Type alias for tool permission callback functions.

```
CanUseTool = Callable[
    [str, dict[str, Any], ToolPermissionContext],
    Awaitable[PermissionResult]
]
```

The callback receives:

- `tool_name` : Name of the tool being called
- `input_data` : The tool's input parameters
- `context` : A `ToolPermissionContext` with additional information

Returns a `PermissionResult` (either `PermissionResultAllow` or `PermissionResultDeny`).

ToolPermissionContext

Context information passed to tool permission callbacks.

```
@dataclass
class ToolPermissionContext:
    signal: Any | None = None # Future: abort signal support
    suggestions: list[PermissionUpdate] =
field(default_factory=list)
    blocked_path: str | None = None
    decision_reason: str | None = None
    title: str | None = None
    display_name: str | None = None
    description: str | None = None
```

Field	Type	Description
signal	Any None	Reserved for future abort signal support
suggestions	list[PermissionUpdate]	Permission update suggestions from the CLI. Bash prompts include a suggestion with the <code>localSettings</code> destination, so returning it in <code>updated_permissions</code> writes the rule to <code>.claude/settings.local.json</code> and persists across sessions.

Field	Type	Description
blocked_path	str None	File path that triggered the permission request, when applicable. For example, when a Bash command tries to access a path outside allowed directories
decision_reason	str None	Reason this permission request was triggered. Forwarded from a PreToolUse hook's permissionDecisionReason when the hook returned "ask"
title	str None	Full permission prompt sentence, such as Claude wants to read foo.txt . Use as the primary prompt text when present
display_name	str None	Short noun phrase for the tool action, such as Read file , suitable for button labels
description	str None	Human-readable subtitle for the permission UI

PermissionResult

Union type for permission callback results.

```
PermissionResult = PermissionResultAllow | PermissionResultDeny
```

PermissionResultAllow

Result indicating the tool call should be allowed.

```
@dataclass
class PermissionResultAllow:
    behavior: Literal["allow"] = "allow"
    updated_input: dict[str, Any] | None = None
    updated_permissions: list[PermissionUpdate] | None = None
```

Field	Type	Default	Description
behavior	Literal["allow"]	"allow"	Must be “allow”
updated_input	dict[str, Any] None	None	Modified input to use instead of original
updated_permission s	list[PermissionUpdate] None	None	Permission updates to apply

PermissionResultDeny

Result indicating the tool call should be denied.

```
@dataclass
class PermissionResultDeny:
    behavior: Literal["deny"] = "deny"
    message: str = ""
    interrupt: bool = False
```

Field	Type	Default	Description
behavior	Literal["deny"]	"deny"	Must be “deny”
message	str	""	Message explaining why the tool was denied
interrupt	bool	False	Whether to interrupt the current execution

PermissionUpdate

Configuration for updating permissions programmatically.

```
@dataclass
class PermissionUpdate:
    type: Literal[
        "addRules",
        "replaceRules",
        "removeRules",
        "setMode",
        "addDirectories",
        "removeDirectories",
    ]
    rules: list[PermissionRuleValue] | None = None
    behavior: Literal["allow", "deny", "ask"] | None = None
    mode: PermissionMode | None = None
    directories: list[str] | None = None
    destination: (
        Literal["userSettings", "projectSettings",
"localSettings", "session"] | None
    ) = None
```

Field	Type	Description
type	Literal[...]	The type of permission update operation
rules	list[PermissionRuleValue] None	Rules for add/replace/remove operations
behavior	Literal["allow", "deny", "ask"] None	Behavior for rule-based operations
mode	PermissionMode None	Mode for setMode operation
directories	list[str] None	Directories for add/remove directory operations
destination	Literal[...] None	Where to apply the permission update

PermissionRuleValue

A rule to add, replace, or remove in a permission update.

```
@dataclass
class PermissionRuleValue:
    tool_name: str
    rule_content: str | None = None
```

ToolsPreset

Preset tools configuration for using Claude Code’s default tool set.

```
class ToolsPreset(TypedDict):
    type: Literal["preset"]
    preset: Literal["claude_code"]
```

ThinkingConfig

Controls extended thinking behavior. A union of three configurations:


```
ThinkingDisplay = Literal["summarized", "omitted"]
```

```
class ThinkingConfigAdaptive(TypedDict):  
    type: Literal["adaptive"]  
    display: NotRequired[ThinkingDisplay]
```

```
class ThinkingConfigEnabled(TypedDict):  
    type: Literal["enabled"]  
    budget_tokens: int  
    display: NotRequired[ThinkingDisplay]
```

```
class ThinkingConfigDisabled(TypedDict):  
    type: Literal["disabled"]
```

```
ThinkingConfig = ThinkingConfigAdaptive | ThinkingConfigEnabled |  
ThinkingConfigDisabled
```

Variant	Fields	Description
adaptive	type , display	Claude adaptively decides when to think
enabled	type , budget_tokens , display	Enable thinking with a specific token budget
disabled	type	Disable thinking

The optional `display` field controls whether thinking text is returned `"summarized"` or `"omitted"` . On Claude Opus 4.7 and later, the API default is `"omitted"` , so set `"summarized"` to receive thinking content in `ThinkingBlock` outputs.

Because these are `TypedDict` classes, they’re plain dicts at runtime. Either construct them as dict literals or call the class like a constructor; both produce a `dict` . Access fields with `config["budget_tokens"]` , not `config.budget_tokens` :

```

from claude_agent_sdk import ClaudeAgentOptions,
ThinkingConfigEnabled

# Option 1: dict literal (recommended, no import needed)
options = ClaudeAgentOptions(thinking={"type": "enabled",
"budget_tokens": 20000})

# Option 2: constructor-style (returns a plain dict)
config = ThinkingConfigEnabled(type="enabled",
budget_tokens=20000)
print(config["budget_tokens"]) # 20000
# config.budget_tokens would raise AttributeError

```

SdkBeta

Literal type for SDK beta features.

```
SdkBeta = Literal["context-1m-2025-08-07"]
```

Use with the `betas` field in `ClaudeAgentOptions` to enable beta features.

⚠ The `context-1m-2025-08-07` beta is retired as of April 30, 2026. Passing this header with Claude Sonnet 4.5 or Sonnet 4 has no effect, and requests that exceed the standard 200k-token context window return an error. To use a 1M-token context window, migrate to **Claude Sonnet 4.6, Claude Opus 4.6, Claude Opus 4.7, or Claude Opus 4.8**, which include 1M context at standard pricing with no beta header required.

McpSdkServerConfig

Configuration for SDK MCP servers created with `create_sdk_mcp_server()`.

```

class McpSdkServerConfig(TypedDict):
    type: Literal["sdk"]
    name: str
    instance: Any # MCP Server instance

```

McpServerConfig

Union type for MCP server configurations.

```
McpServerConfig = (  
    McpStdioServerConfig | McpSSEServerConfig |  
    McpHttpServerConfig | McpSdkServerConfig  
)
```

McpStdioServerConfig

```
class McpStdioServerConfig(TypedDict):  
    type: NotRequired[Literal["stdio"]] # Optional for backwards  
compatibility  
    command: str  
    args: NotRequired[list[str]]  
    env: NotRequired[dict[str, str]]
```

McpSSEServerConfig

```
class McpSSEServerConfig(TypedDict):  
    type: Literal["sse"]  
    url: str  
    headers: NotRequired[dict[str, str]]
```

McpHttpServerConfig

```
class McpHttpServerConfig(TypedDict):  
    type: Literal["http"]  
    url: str  
    headers: NotRequired[dict[str, str]]
```

McpServerStatusConfig

The configuration of an MCP server as reported by `get_mcp_status()`. This is the union of all `McpServerConfig` transport variants plus an output-only `claudeai-proxy` variant for servers proxied through claude.ai.

```
McpServerStatusConfig = (  
    McpStdioServerConfig  
    | McpSSEServerConfig  
    | McpHttpServerConfig  
    | McpSdkServerConfigStatus  
    | McpClaudeAIProxyServerConfig  
)
```

`McpSdkServerConfigStatus` is the serializable form of `McpSdkServerConfig` with only `type` (`"sdk"`) and `name` (`str`) fields; the in-process `instance` is omitted.

`McpClaudeAIProxyServerConfig` has `type` (`"claudeai-proxy"`), `url` (`str`), and `id` (`str`) fields.

McpStatusResponse

Response from `ClaudeSDKClient.get_mcp_status()` . Wraps the list of server statuses under the `mcpServers` key.

```
class McpStatusResponse(TypedDict):  
    mcpServers: list[McpServerStatus]
```

McpServerStatus

Status of a connected MCP server, contained in `McpStatusResponse` .

```
class McpServerStatus(TypedDict):  
    name: str  
    status: McpServerConnectionStatus # "connected" | "failed" |  
    "needs-auth" | "pending" | "disabled"  
    serverInfo: NotRequired[McpServerInfo]  
    error: NotRequired[str]  
    config: NotRequired[McpServerStatusConfig]  
    scope: NotRequired[str]  
    tools: NotRequired[list[McpToolInfo]]
```

Field	Type	Description
name	str	Server name

Field	Type	Description
status	str	One of "connected" , "failed" , "needs-auth" , "pending" , or "disabled"
serverInfo	dict (optional)	Server name and version ({ "name": str, "version": str})
error	str (optional)	Error message if the server failed to connect
config	<u>McpServerStatusConfig</u> (optional)	Server configuration. Same shape as <u>McpServerConfig</u> (stdio, SSE, HTTP, or SDK), plus a <code>claudeai-proxy</code> variant for servers connected through claude.ai
scope	str (optional)	Configuration scope
tools	list (optional)	Tools provided by this server, each with <code>name</code> , <code>description</code> , and <code>annotations</code> fields

SdkPluginConfig

Configuration for loading plugins in the SDK.

```
class SdkPluginConfig(TypedDict):
    type: Literal["local"]
    path: str
```

Field	Type	Description
type	Literal["local"]	Must be "local" (only local plugins currently supported)
path	str	Absolute or relative path to the plugin directory

Example:

```
plugins = [
    {"type": "local", "path": "./my-plugin"},
    {"type": "local", "path": "/absolute/path/to/plugin"},
]
```

For complete information on creating and using plugins, see [Plugins](#).

Message Types

Message

Union type of all possible messages.

```
Message = (  
    UserMessage  
    | AssistantMessage  
    | SystemMessage  
    | ResultMessage  
    | StreamEvent  
    | RateLimitEvent  
)
```

UserMessage

User input message.

```
@dataclass  
class UserMessage:  
    content: str | list[ContentBlock]  
    uuid: str | None = None  
    parent_tool_use_id: str | None = None  
    tool_use_result: dict[str, Any] | None = None
```

Field	Type	Description
content	str list[ContentBlock]	Message content as text or content blocks
uuid	str None	Unique message identifier
parent_tool_use_id	str None	Tool use ID if this message is a tool result response
tool_use_result	dict[str, Any] None	Tool result data if applicable

AssistantMessage

Assistant response message with content blocks.

```
@dataclass
class AssistantMessage:
    content: list[ContentBlock]
    model: str
    parent_tool_use_id: str | None = None
    error: AssistantMessageError | None = None
    usage: dict[str, Any] | None = None
    message_id: str | None = None
```

Field	Type	Description
content	list[ContentBlock]	List of content blocks in the response
model	str	Model that generated the response
parent_tool_use_id	str None	Tool use ID if this is a nested response
error	<u>AssistantMessageError</u> None	Error type if the response encountered an error
usage	dict[str, Any] None	Per-message token usage (same keys as <u>ResultMessage.usage</u>)
message_id	str None	API message ID. Multiple messages from one turn share the same ID

AssistantMessageError

Possible error types for assistant messages.

```
AssistantMessageError = Literal[
    "authentication_failed",
    "billing_error",
    "rate_limit",
    "invalid_request",
    "server_error",
    "max_output_tokens",
    "unknown",
]
```

SystemMessage

System message with metadata.

```
@dataclass
class SystemMessage:
    subtype: str
    data: dict[str, Any]
```

ResultMessage

Final result message with cost and usage information.

```
@dataclass
class ResultMessage:
    subtype: str
    duration_ms: int
    duration_api_ms: int
    is_error: bool
    num_turns: int
    session_id: str
    stop_reason: str | None = None
    total_cost_usd: float | None = None
    usage: dict[str, Any] | None = None
    result: str | None = None
    structured_output: Any = None
    model_usage: dict[str, Any] | None = None
    permission_denials: list[Any] | None = None
    deferred_tool_use: DeferredToolUse | None = None
    errors: list[str] | None = None
    api_error_status: int | None = None
    uuid: str | None = None
```

The `subtype` field determines which other fields are populated. It is one of `"success"`, `"error_during_execution"`, `"error_max_turns"`, `"error_max_budget_usd"`, or `"error_max_structured_output_retries"`. The Python dataclass flattens all variants into one shape, so fields that don't apply to the returned subtype are `None`.

Several fields carry diagnostic detail when the conversation ends on an error:

- `is_error` : `True` when the conversation ended in an error state. Always `True` on the

`error_*` subtypes. On `subtype="success"` it is `True` when the final model request failed, meaning the agent loop completed but the last API call returned an error.

- `api_error_status` : the HTTP status code of the terminating API error. `None` when the turn ended without one. Populated only on `subtype="success"` .
- `result` : text of the final assistant message on `subtype="success"` , or `None` on the `error_*` subtypes. When `subtype="success"` and `is_error=True` , this holds the API error string if one is available but can be empty, so check `api_error_status` and the preceding `AssistantMessage` content for detail.
- `errors` : loop-level error strings such as the max-turns message. Populated only on the `error_*` subtypes.

The `usage` dict contains the following keys when present:

Key	Type	Description
<code>input_tokens</code>	<code>int</code>	Total input tokens consumed.
<code>output_tokens</code>	<code>int</code>	Total output tokens generated.
<code>cache_creation_input_tokens</code>	<code>int</code>	Tokens used to create new cache entries.
<code>cache_read_input_tokens</code>	<code>int</code>	Tokens read from existing cache entries.

The `model_usage` dict maps model names to per-model usage. The inner dict keys use camelCase because the value is passed through unmodified from the underlying CLI process, matching the TypeScript `ModelUsage` type:

Key	Type	Description
inputTokens	int	Input tokens for this model.
outputTokens	int	Output tokens for this model.
cacheReadInputTokens	int	Cache read tokens for this model.
cacheCreationInputTokens	int	Cache creation tokens for this model.
webSearchRequests	int	Web search requests made by this model.
costUSD	float	Estimated cost in USD for this model, computed client-side. See Track cost and usage for billing caveats.
contextWindow	int	Context window size for this model.
maxOutputTokens	int	Maximum output token limit for this model.

StreamEvent

Stream event for partial message updates during streaming. Only received when

```
include_partial_messages=True in ClaudeAgentOptions . Import via from
claude_agent_sdk.types import StreamEvent .
```

```
@dataclass
class StreamEvent:
    uuid: str
    session_id: str
    event: dict[str, Any] # The raw Claude API stream event
    parent_tool_use_id: str | None = None
```

Field	Type	Description
uuid	str	Unique identifier for this event
session_id	str	Session identifier
event	dict[str, Any]	The raw Claude API stream event data
parent_tool_use_id	str None	Parent tool use ID if this event is from a subagent

RateLimitEvent

Emitted when rate limit status changes (for example, from "allowed" to "allowed_warning"). Use this to warn users before they hit a hard limit, or to back off when status is "rejected" .

```
@dataclass
class RateLimitEvent:
    rate_limit_info: RateLimitInfo
    uuid: str
    session_id: str
```

Field	Type	Description
rate_limit_info	<u>RateLimitInfo</u>	Current rate limit state
uuid	str	Unique event identifier
session_id	str	Session identifier

RateLimitInfo

Rate limit state carried by RateLimitEvent .

```
RateLimitStatus = Literal["allowed", "allowed_warning",
"rejected"]
RateLimitType = Literal[
    "five_hour", "seven_day", "seven_day_opus",
    "seven_day_sonnet", "overage"
]
```

```
@dataclass
class RateLimitInfo:
    status: RateLimitStatus
    resets_at: int | None = None
    rate_limit_type: RateLimitType | None = None
    utilization: float | None = None
    overage_status: RateLimitStatus | None = None
    overage_resets_at: int | None = None
    overage_disabled_reason: str | None = None
    raw: dict[str, Any] = field(default_factory=dict)
```

Field	Type	Description
status	RateLimitStatus	Current status. "allowed_warning" means approaching the limit; "rejected" means the limit was hit
resets_at	int None	Unix timestamp when the rate limit window resets
rate_limit_type	RateLimitType None	Which rate limit window applies
utilization	float None	Fraction of the rate limit consumed (0.0 to 1.0)
overage_status	RateLimitStatus None	Status of pay-as-you-go overage usage, if applicable
overage_resets_at	int None	Unix timestamp when the overage window resets
overage_disabled_reason	str None	Why overage is unavailable, if status is "rejected"
raw	dict[str, Any]	Full raw dict from the CLI, including fields not modeled above

TaskStartedMessage

Emitted when a background task starts. A background task is anything tracked outside the main turn: a backgrounded Bash command, a **Monitor** watch, a subagent spawned via the Agent tool, or a remote agent. The `task_type` field tells you which. This naming is unrelated to the `Task`-to-`Agent` tool rename.

```
@dataclass
class TaskStartedMessage(SystemMessage):
    task_id: str
    description: str
    uuid: str
    session_id: str
    tool_use_id: str | None = None
    task_type: str | None = None
```

Field	Type	Description
task_id	str	Unique identifier for the task
description	str	Description of the task
uuid	str	Unique message identifier

Field	Type	Description
session_id	str	Session identifier
tool_use_id	str None	Associated tool use ID
task_type	str None	Which kind of background task: "local_bash" for background Bash and Monitor watches, "local_agent" , Or "remote_agent"

TaskUsage

Token and timing data for a background task.

```
class TaskUsage(TypedDict):
    total_tokens: int
    tool_uses: int
    duration_ms: int
```

TaskProgressMessage

Emitted periodically with progress updates for a running background task.

```
@dataclass
class TaskProgressMessage(SystemMessage):
    task_id: str
    description: str
    usage: TaskUsage
    uuid: str
    session_id: str
    tool_use_id: str | None = None
    last_tool_name: str | None = None
```

Field	Type	Description
task_id	str	Unique identifier for the task
description	str	Current status description
usage	TaskUsage	Token usage for this task so far
uuid	str	Unique message identifier
session_id	str	Session identifier

Field	Type	Description
tool_use_id	str None	Associated tool use ID
last_tool_name	str None	Name of the last tool the task used

TaskNotificationMessage

Emitted when a background task completes, fails, or is stopped. Background tasks include `run_in_background` Bash commands, Monitor watches, and background subagents.

```
@dataclass
class TaskNotificationMessage(SystemMessage):
    task_id: str
    status: TaskNotificationStatus # "completed" | "failed" |
    "stopped"
    output_file: str
    summary: str
    uuid: str
    session_id: str
    tool_use_id: str | None = None
    usage: TaskUsage | None = None
```

Field	Type	Description
task_id	str	Unique identifier for the task
status	TaskNotificationStatus	One of "completed" , "failed" , or "stopped"
output_file	str	Path to the task output file
summary	str	Summary of the task result
uuid	str	Unique message identifier
session_id	str	Session identifier
tool_use_id	str None	Associated tool use ID
usage	TaskUsage None	Final token usage for the task

Content Block Types

ContentBlock

Union type of all content blocks.

```
ContentBlock = TextBlock | ThinkingBlock | ToolUseBlock |  
ToolResultBlock
```

TextBlock

Text content block.

```
@dataclass  
class TextBlock:  
    text: str
```

ThinkingBlock

Thinking content block (for models with thinking capability).

```
@dataclass  
class ThinkingBlock:  
    thinking: str  
    signature: str
```

ToolUseBlock

Tool use request block.

```
@dataclass  
class ToolUseBlock:  
    id: str  
    name: str  
    input: dict[str, Any]
```

ToolResultBlock

Tool execution result block.

```
@dataclass
class ToolResultBlock:
    tool_use_id: str
    content: str | list[dict[str, Any]] | None = None
    is_error: bool | None = None
```

Error Types

ClaudeSDKError

Base exception class for all SDK errors.

```
class ClaudeSDKError(Exception):
    """Base error for Claude SDK."""
```

CLINotFoundError

Raised when Claude Code CLI is not installed or not found.

```
class CLINotFoundError(CLIConnectionError):
    def __init__(
        self, message: str = "Claude Code not found", cli_path:
str | None = None
    ):
        """
        Args:
            message: Error message (default: "Claude Code not
found")
            cli_path: Optional path to the CLI that was not found
        """
```

CLIConnectionError

Raised when connection to Claude Code fails.


```
class CLIConnectionError(ClaudeSDKError):
    """Failed to connect to Claude Code."""
```

ProcessError

Raised when the Claude Code process fails.

```
class ProcessError(ClaudeSDKError):
    def __init__(
        self, message: str, exit_code: int | None = None, stderr:
str | None = None
    ):
        self.exit_code = exit_code
        self.stderr = stderr
```

CLIJSONDecodeError

Raised when JSON parsing fails.

```
class CLIJSONDecodeError(ClaudeSDKError):
    def __init__(self, line: str, original_error: Exception):
        """
        Args:
            line: The line that failed to parse
            original_error: The original JSON decode exception
        """
        self.line = line
        self.original_error = original_error
```

Hook Types

For a comprehensive guide on using hooks with examples and common patterns, see the [Hooks guide](#).

HookEvent

Supported hook event types.

```
HookEvent = Literal[
    "PreToolUse", # Called before tool execution
    "PostToolUse", # Called after tool execution
    "PostToolUseFailure", # Called when a tool execution fails
    "UserPromptSubmit", # Called when user submits a prompt
    "Stop", # Called when stopping execution
    "SubagentStop", # Called when a subagent stops
    "PreCompact", # Called before message compaction
    "Notification", # Called for notification events
    "SubagentStart", # Called when a subagent starts
    "PermissionRequest", # Called when a permission decision is
needed
]
```

❗ The TypeScript SDK supports additional hook events not yet available in Python: `SessionStart` , `SessionEnd` , `Setup` , `TeammateIdle` , `TaskCompleted` , `ConfigChange` , `WorktreeCreate` , `WorktreeRemove` , `PostToolBatch` , and `MessageDisplay` .

HookCallback

Type definition for hook callback functions.

```
HookCallback = Callable[[HookInput, str | None, HookContext],
Awaitable[HookJSONOutput]]
```

Parameters:

- `input` : Strongly-typed hook input with discriminated unions based on `hook_event_name` (see `HookInput`)
- `tool_use_id` : Optional tool use identifier (for tool-related hooks)
- `context` : Hook context with additional information

Returns a `HookJSONOutput` that may contain:

- `decision` : `"block"` to block the action
- `systemMessage` : warning message shown to the user
- `hookSpecificOutput` : Hook-specific output data

HookContext

Context information passed to hook callbacks.

```
class HookContext(TypedDict):  
    signal: Any | None # Future: abort signal support
```

HookMatcher

Configuration for matching hooks to specific events or tools.

```
@dataclass  
class HookMatcher:  
    matcher: str | None = (  
        None # Tool name or pattern to match (e.g., "Bash",  
        "Write|Edit")  
    )  
    hooks: list[HookCallback] = field(  
        default_factory=list  
    ) # List of callbacks to execute  
    timeout: float | None = (  
        None # Timeout in seconds for all hooks in this matcher  
        (default: 60)  
    )
```

HookInput

Union type of all hook input types. The actual type depends on the `hook_event_name` field.

```
HookInput = (
    PreToolUseHookInput
    | PostToolUseHookInput
    | PostToolUseFailureHookInput
    | UserPromptSubmitHookInput
    | StopHookInput
    | SubagentStopHookInput
    | PreCompactHookInput
    | NotificationHookInput
    | SubagentStartHookInput
    | PermissionRequestHookInput
)
```

BaseHookInput

Base fields present in all hook input types.

```
class BaseHookInput(TypedDict):
    session_id: str
    transcript_path: str
    cwd: str
    permission_mode: NotRequired[str]
```

Field	Type	Description
session_id	str	Current session identifier
transcript_path	str	Path to the session transcript file
cwd	str	Current working directory
permission_mode	str (optional)	Current permission mode

PreToolUseHookInput

Input data for PreToolUse hook events.

```

class PreToolUseHookInput(BaseHookInput):
    hook_event_name: Literal["PreToolUse"]
    tool_name: str
    tool_input: dict[str, Any]
    tool_use_id: str
    agent_id: NotRequired[str]
    agent_type: NotRequired[str]

```

Field	Type	Description
hook_event_name	Literal["PreToolUse"]	Always “PreToolUse”
tool_name	str	Name of the tool about to be executed
tool_input	dict[str, Any]	Input parameters for the tool
tool_use_id	str	Unique identifier for this tool use
agent_id	str (optional)	Subagent identifier, present when the hook fires inside a subagent
agent_type	str (optional)	Subagent type, present when the hook fires inside a subagent

PostToolUseHookInput

Input data for `PostToolUse` hook events.

```

class PostToolUseHookInput(BaseHookInput):
    hook_event_name: Literal["PostToolUse"]
    tool_name: str
    tool_input: dict[str, Any]
    tool_response: Any
    tool_use_id: str
    agent_id: NotRequired[str]
    agent_type: NotRequired[str]

```

Field	Type	Description
hook_event_name	Literal["PostToolUse"]	Always “PostToolUse”
tool_name	str	Name of the tool that was executed

Field	Type	Description
tool_input	dict[str, Any]	Input parameters that were used
tool_response	Any	Response from the tool execution
tool_use_id	str	Unique identifier for this tool use
agent_id	str (optional)	Subagent identifier, present when the hook fires inside a subagent
agent_type	str (optional)	Subagent type, present when the hook fires inside a subagent

PostToolUseFailureHookInput

Input data for `PostToolUseFailure` hook events. Called when a tool execution fails.

```
class PostToolUseFailureHookInput(BaseHookInput):
    hook_event_name: Literal["PostToolUseFailure"]
    tool_name: str
    tool_input: dict[str, Any]
    tool_use_id: str
    error: str
    is_interrupt: NotRequired[bool]
    agent_id: NotRequired[str]
    agent_type: NotRequired[str]
```

Field	Type	Description
hook_event_name	Literal["PostToolUseFailure"]	Always “PostToolUseFailure”
tool_name	str	Name of the tool that failed
tool_input	dict[str, Any]	Input parameters that were used
tool_use_id	str	Unique identifier for this tool use
error	str	Error message from the failed execution
is_interrupt	bool (optional)	Whether the failure was caused by an interrupt
agent_id	str (optional)	Subagent identifier, present when the hook fires inside a subagent
agent_type	str (optional)	Subagent type, present when the hook fires inside a subagent

UserPromptSubmitHookInput

Input data for `UserPromptSubmit` hook events.

```
class UserPromptSubmitHookInput(BaseHookInput):
    hook_event_name: Literal["UserPromptSubmit"]
    prompt: str
```

Field	Type	Description
hook_event_name	Literal["UserPromptSubmit"]	Always “UserPromptSubmit”
prompt	str	The user’s submitted prompt

StopHookInput

Input data for `Stop` hook events.

```
class StopHookInput(BaseHookInput):
    hook_event_name: Literal["Stop"]
    stop_hook_active: bool
```

Field	Type	Description
hook_event_name	Literal["Stop"]	Always “Stop”
stop_hook_active	bool	Whether the stop hook is active

SubagentStopHookInput

Input data for `SubagentStop` hook events.

```
class SubagentStopHookInput(BaseHookInput):
    hook_event_name: Literal["SubagentStop"]
    stop_hook_active: bool
    agent_id: str
    agent_transcript_path: str
    agent_type: str
```

Field	Type	Description
hook_event_name	Literal["SubagentStop"]	Always “SubagentStop”
stop_hook_active	bool	Whether the stop hook is active
agent_id	str	Unique identifier for the subagent
agent_transcript_path	str	Path to the subagent’s transcript file
agent_type	str	Type of the subagent

PreCompactHookInput

Input data for `PreCompact` hook events.

```
class PreCompactHookInput(BaseHookInput):
    hook_event_name: Literal["PreCompact"]
    trigger: Literal["manual", "auto"]
    custom_instructions: str | None
```

Field	Type	Description
hook_event_name	Literal["PreCompact"]	Always “PreCompact”
trigger	Literal["manual", "auto"]	What triggered the compaction
custom_instructions	str None	Custom instructions for compaction

NotificationHookInput

Input data for `Notification` hook events.

```
class NotificationHookInput(BaseHookInput):
    hook_event_name: Literal["Notification"]
    message: str
    title: NotRequired[str]
    notification_type: str
```


Field	Type	Description
hook_event_name	Literal["Notification"]	Always “Notification”
message	str	Notification message content
title	str (optional)	Notification title
notification_type	str	Type of notification

SubagentStartHookInput

Input data for SubagentStart hook events.

```
class SubagentStartHookInput(BaseHookInput):
    hook_event_name: Literal["SubagentStart"]
    agent_id: str
    agent_type: str
```

Field	Type	Description
hook_event_name	Literal["SubagentStart"]	Always “SubagentStart”
agent_id	str	Unique identifier for the subagent
agent_type	str	Type of the subagent

PermissionRequestHookInput

Input data for PermissionRequest hook events. Allows hooks to handle permission decisions programmatically.

```
class PermissionRequestHookInput(BaseHookInput):
    hook_event_name: Literal["PermissionRequest"]
    tool_name: str
    tool_input: dict[str, Any]
    permission_suggestions: NotRequired[list[Any]]
```

Field	Type	Description
hook_event_name	Literal["PermissionRequest"]	Always "PermissionRequest"
tool_name	str	Name of the tool requesting permission
tool_input	dict[str, Any]	Input parameters for the tool
permission_suggestions	list[Any] (optional)	Suggested permission updates from the CLI

HookJSONOutput

Union type for hook callback return values.

```
HookJSONOutput = AsyncHookJSONOutput | SyncHookJSONOutput
```

SyncHookJSONOutput

Synchronous hook output with control and decision fields.

```
class SyncHookJSONOutput(TypedDict):
    # Control fields
    continue_: NotRequired[bool] # Whether to proceed (default:
    True)
    suppressOutput: NotRequired[bool] # Hide stdout from
    transcript
    stopReason: NotRequired[str] # Message when continue is
    False

    # Decision fields
    decision: NotRequired[Literal["block"]]
    systemMessage: NotRequired[str] # Warning message for user
    reason: NotRequired[str] # Feedback for Claude

    # Hook-specific output
    hookSpecificOutput: NotRequired[HookSpecificOutput]
```

⚠ Use `continue_` (with underscore) in Python code. It is automatically converted to `continue` when sent to the CLI.

HookSpecificOutput

A `TypedDict` containing the hook event name and event-specific fields. The shape depends on the `hookEventName` value. For full details on available fields per hook event, see [**Control execution with hooks**](#).

A discriminated union of event-specific output types. The `hookEventName` field determines which fields are valid.

```
class PreToolUseHookSpecificOutput(TypedDict):
    hookEventName: Literal["PreToolUse"]
    permissionDecision: NotRequired[Literal["allow", "deny",
"ask", "defer"]]
    permissionDecisionReason: NotRequired[str]
    updatedInput: NotRequired[dict[str, Any]]
    additionalContext: NotRequired[str]
```

```
class PostToolUseHookSpecificOutput(TypedDict):
    hookEventName: Literal["PostToolUse"]
    additionalContext: NotRequired[str]
    updatedToolOutput: NotRequired[Any]
    updatedMCPToolOutput: NotRequired[Any] # Deprecated: use
updatedToolOutput, which works for all tools
```

```
class PostToolUseFailureHookSpecificOutput(TypedDict):
    hookEventName: Literal["PostToolUseFailure"]
    additionalContext: NotRequired[str]
```

```
class UserPromptSubmitHookSpecificOutput(TypedDict):
    hookEventName: Literal["UserPromptSubmit"]
    additionalContext: NotRequired[str]
```

```
class NotificationHookSpecificOutput(TypedDict):
    hookEventName: Literal["Notification"]
    additionalContext: NotRequired[str]
```

```
class SubagentStartHookSpecificOutput(TypedDict):
    hookEventName: Literal["SubagentStart"]
    additionalContext: NotRequired[str]
```

```
class PermissionRequestHookSpecificOutput(TypedDict):
    hookEventName: Literal["PermissionRequest"]
    decision: dict[str, Any]
```

```
HookSpecificOutput = (
    PreToolUseHookSpecificOutput
```

```
| PostToolUseHookSpecificOutput
| PostToolUseFailureHookSpecificOutput
| UserPromptSubmitHookSpecificOutput
| NotificationHookSpecificOutput
| SubagentStartHookSpecificOutput
| PermissionRequestHookSpecificOutput
)
```

AsyncHookJSONOutput

Async hook output that defers hook execution.

```
class AsyncHookJSONOutput(TypedDict):
    async_: Literal[True] # Set to True to defer execution
    asyncTimeout: NotRequired[int] # Timeout in milliseconds
```

ⓘ Use `async_` (with underscore) in Python code. It is automatically converted to `async` when sent to the CLI.

Hook Usage Example

This example registers two hooks: one that blocks dangerous bash commands like `rm -rf /`, and another that logs all tool usage for auditing. The security hook only runs on Bash commands (via the `matcher`), while the logging hook runs on all tools.

```

from claude_agent_sdk import query, ClaudeAgentOptions,
HookMatcher, HookContext
from typing import Any

async def validate_bash_command(
    input_data: dict[str, Any], tool_use_id: str | None, context:
HookContext
) -> dict[str, Any]:
    """Validate and potentially block dangerous bash commands."""
    if input_data["tool_name"] == "Bash":
        command = input_data["tool_input"].get("command", "")
        if "rm -rf /" in command:
            return {
                "hookSpecificOutput": {
                    "hookEventName": "PreToolUse",
                    "permissionDecision": "deny",
                    "permissionDecisionReason": "Dangerous
command blocked",
                }
            }
        return {}

async def log_tool_use(
    input_data: dict[str, Any], tool_use_id: str | None, context:
HookContext
) -> dict[str, Any]:
    """Log all tool usage for auditing."""
    print(f"Tool used: {input_data.get('tool_name')}")
    return {}

options = ClaudeAgentOptions(
    hooks={
        "PreToolUse": [
            HookMatcher(
                matcher="Bash", hooks=[validate_bash_command],
timeout=120
            ), # 2 min for validation
            HookMatcher(
                hooks=[log_tool_use]
            ), # Applies to all tools (default 60s timeout)
    ],

```

```

        "PostToolUse": [HookMatcher(hooks=[log_tool_use])],
    }
)

async for message in query(prompt="Analyze this codebase",
    options=options):
    print(message)

```

Tool Input/Output Types

Documentation of input/output schemas for all built-in Claude Code tools. While the Python SDK doesn't export these as types, they represent the structure of tool inputs and outputs in messages.

Agent

Tool name: `Agent` (previously `Task`, which is still accepted as an alias)

Input:

```

{
    "description": str, # A short (3-5 word) description of the
task
    "prompt": str, # The task for the agent to perform
    "subagent_type": str, # The type of specialized agent to use
}

```

Output:

```

{
    "result": str, # Final result from the subagent
    "usage": dict | None, # Token usage statistics
    "total_cost_usd": float | None, # Estimated total cost in
USD
    "duration_ms": int | None, # Execution duration in
milliseconds
}

```

AskUserQuestion

Tool name: AskUserQuestion

Asks the user clarifying questions during execution. See [Handle approvals and user input](#) for usage details.

Input:

```
{
  "questions": [ # Questions to ask the user (1-4 questions)
    {
      "question": str, # The complete question to ask the
user
      "header": str, # Very short label displayed as a
chip/tag (max 12 chars)
      "options": [ # The available choices (2-4 options)
        {
          "label": str, # Display text for this option
(1-5 words)
          "description": str, # Explanation of what
this option means
        }
      ],
      "multiSelect": bool, # Set to true to allow multiple
selections
    }
  ],
  "answers": dict[str, str | list[str]] | None,
  # User answers populated by the permission system. Multi-
select
  # answers may be a list of labels or a comma-joined string
}
```

Output:


```
{
  "questions": [ # The questions that were asked
    {
      "question": str,
      "header": str,
      "options": [{"label": str, "description": str}],
      "multiSelect": bool,
    }
  ],
  "answers": dict[str, str], # Maps question text to answer
string
  # Multi-select answers are comma-separated
}
```

Bash

Tool name: Bash

Input:

```
{
  "command": str, # The command to execute
  "timeout": int | None, # Optional timeout in milliseconds
(max 600000)
  "description": str | None, # Clear, concise description (5-
10 words)
  "run_in_background": bool | None, # Set to true to run in
background
}
```

Output:

```
{
  "output": str, # Combined stdout and stderr output
  "exitCode": int, # Exit code of the command
  "killed": bool | None, # Whether command was killed due to
timeout
  "shellId": str | None, # Shell ID for background processes
}
```

Monitor

Tool name: Monitor

Runs a background script and delivers each stdout line to Claude as an event so it can react without polling. Monitor follows the same permission rules as Bash. See the [Monitor tool reference](#) for behavior and provider availability.

Input:

```
{
  "command": str, # Shell script; each stdout line is an
event, exit ends the watch
  "description": str, # Short description shown in
notifications
  "timeout_ms": int | None, # Kill after this deadline
(default 300000, max 3600000)
  "persistent": bool | None, # Run for the lifetime of the
session; stop with TaskStop
}
```

Output:

```
{
  "taskId": str, # ID of the background monitor task
  "timeoutMs": int, # Timeout deadline in milliseconds (0 when
persistent)
  "persistent": bool | None, # True when running until
TaskStop or session end
}
```

Edit

Tool name: Edit

Input:

```
{
  "file_path": str, # The absolute path to the file to modify
  "old_string": str, # The text to replace
  "new_string": str, # The text to replace it with
  "replace_all": bool | None, # Replace all occurrences
  (default False)
}
```

Output:

```
{
  "message": str, # Confirmation message
  "replacements": int, # Number of replacements made
  "file_path": str, # File path that was edited
}
```

Read

Tool name: Read

Input:

```
{
  "file_path": str, # The absolute path to the file to read
  "offset": int | None, # The line number to start reading
  from
  "limit": int | None, # The number of lines to read
}
```

Output (Text files):

```
{
  "content": str, # File contents with line numbers
  "total_lines": int, # Total number of lines in file
  "lines_returned": int, # Lines actually returned
}
```

Output (Images):

```
{
  "image": str, # Base64 encoded image data
  "mime_type": str, # Image MIME type
  "file_size": int, # File size in bytes
}
```

Write

Tool name: Write

Input:

```
{
  "file_path": str, # The absolute path to the file to write
  "content": str, # The content to write to the file
}
```

Output:

```
{
  "message": str, # Success message
  "bytes_written": int, # Number of bytes written
  "file_path": str, # File path that was written
}
```

Glob

Tool name: Glob

Input:

```
{
  "pattern": str, # The glob pattern to match files against
  "path": str | None, # The directory to search in (defaults
to cwd)
}
```

Output:

```
{  
    "matches": list[str], # Array of matching file paths  
    "count": int, # Number of matches found  
    "search_path": str, # Search directory used  
}
```

Grep

Tool name: Grep

Input:

```
{  
    "pattern": str, # The regular expression pattern  
    "path": str | None, # File or directory to search in  
    "glob": str | None, # Glob pattern to filter files  
    "type": str | None, # File type to search  
    "output_mode": str | None, # "content",  
    "files_with_matches", or "count"  
    "-i": bool | None, # Case insensitive search  
    "-n": bool | None, # Show line numbers  
    "-B": int | None, # Lines to show before each match  
    "-A": int | None, # Lines to show after each match  
    "-C": int | None, # Lines to show before and after  
    "head_limit": int | None, # Limit output to first N  
    lines/entries  
    "multiline": bool | None, # Enable multiline mode  
}
```

Output (content mode):

```
{
    "matches": [
        {
            "file": str,
            "line_number": int | None,
            "line": str,
            "before_context": list[str] | None,
            "after_context": list[str] | None,
        }
    ],
    "total_matches": int,
}
```

Output (files_with_matches mode):

```
{
    "files": list[str], # Files containing matches
    "count": int, # Number of files with matches
}
```

NotebookEdit

Tool name: NotebookEdit

Input:

```
{
    "notebook_path": str, # Absolute path to the Jupyter
notebook
    "cell_id": str | None, # The ID of the cell to edit
    "new_source": str, # The new source for the cell
    "cell_type": "code" | "markdown" | None, # The type of the
cell
    "edit_mode": "replace" | "insert" | "delete" | None, # Edit
operation type
}
```

Output:

```
{
  "message": str, # Success message
  "edit_type": "replaced" | "inserted" | "deleted", # Type of
edit performed
  "cell_id": str | None, # Cell ID that was affected
  "total_cells": int, # Total cells in notebook after edit
}
```

WebFetch

Tool name: WebFetch

Input:

```
{
  "url": str, # The URL to fetch content from
  "prompt": str, # The prompt to run on the fetched content
}
```

Output:

```
{
  "bytes": int, # Size of the fetched content in bytes
  "code": int, # HTTP response code
  "codeText": str, # HTTP response code text
  "result": str, # Processed result from applying the prompt
to the content
  "durationMs": int, # Time to fetch and process the content,
in milliseconds
  "url": str, # URL that was fetched
}
```

WebSearch

Tool name: WebSearch

Input:

```
{
  "query": str, # The search query to use
  "allowed_domains": list[str] | None, # Only include results
from these domains
  "blocked_domains": list[str] | None, # Never include results
from these domains
}
```

Output:

```
{
  "query": str, # The search query
  "results": list[str | {"tool_use_id": str, "content":
list[{"title": str, "url": str}]]],
  "durationSeconds": float, # Search duration in seconds
}
```

TodoWrite

Tool name: TodoWrite

ⓘ As of Claude Code v2.1.142, `TodoWrite` is disabled by default. Use `TaskCreate` , `TaskGet` , `TaskUpdate` , and `TaskList` instead. See [Migrate to Task tools](#) to update your monitoring code, or set `CLAUDE_CODE_ENABLE_TASKS=0` to revert to `TodoWrite` .

Input:

```
{
  "todos": [
    {
      "content": str, # The task description
      "status": "pending" | "in_progress" | "completed", #
Task status
      "activeForm": str, # Active form of the description
    }
  ]
}
```

Output:


```
{
  "message": str, # Success message
  "stats": {"total": int, "pending": int, "in_progress": int,
"completed": int},
}
```

TaskCreate

Tool name: TaskCreate

Input:

```
{
  "subject": str, # Short task title
  "description": str, # Detailed task body
  "activeForm": str | None, # Present-tense label shown while
in progress
  "metadata": dict | None, # Arbitrary caller metadata
}
```

Output:

```
{
  "task": {"id": str, "subject": str}, # Created task with
assigned ID
}
```

TaskUpdate

Tool name: TaskUpdate

Input:

```
{
  "taskId": str, # ID of the task to patch
  "status": Literal["pending", "in_progress", "completed",
"deleted"] | None,
  "subject": str | None,
  "description": str | None,
  "activeForm": str | None,
  "addBlocks": list[str] | None, # Task IDs this task now
blocks
  "addBlockedBy": list[str] | None, # Task IDs that now block
this task
  "owner": str | None,
  "metadata": dict | None,
}
```

Output:

```
{
  "success": bool,
  "taskId": str,
  "updatedFields": list[str], # Names of fields that changed
  "error": str | None,
  "statusChange": {"from": str, "to": str} | None,
}
```

TaskGet

Tool name: TaskGet

Input:

```
{
  "taskId": str, # ID of the task to read
}
```

Output:

```
{
  "task": {
    "id": str,
    "subject": str,
    "description": str,
    "status": Literal["pending", "in_progress", "completed"],
    "blocks": list[str],
    "blockedBy": list[str],
  } | None, # None when the ID is not found
}
```

TaskList

Tool name: TaskList

Input:

```
{}
```

Output:

```
{
  "tasks": [
    {
      "id": str,
      "subject": str,
      "status": Literal["pending", "in_progress",
"completed"],
      "owner": str | None,
      "blockedBy": list[str],
    }
  ],
}
```

BashOutput

Tool name: BashOutput

Input:

```
{
  "bash_id": str, # The ID of the background shell
  "filter": str | None, # Optional regex to filter output
lines
}
```

Output:

```
{
  "output": str, # New output since last check
  "status": "running" | "completed" | "failed", # Current
shell status
  "exitCode": int | None, # Exit code when completed
}
```

KillBash

Tool name: KillBash

Input:

```
{
  "shell_id": str # The ID of the background shell to kill
}
```

Output:

```
{
  "message": str, # Success message
  "shell_id": str, # ID of the killed shell
}
```

ExitPlanMode

Tool name: ExitPlanMode

Input:

```
{
  "plan": str # The plan to run by the user for approval
}
```

Output:

```
{
  "message": str, # Confirmation message
  "approved": bool | None, # Whether user approved the plan
}
```

ListMcpResources

Tool name: ListMcpResourcesTool

Input:

```
{
  "server": str | None # Optional server name to filter
resources by
}
```

Output:

```
{
  "resources": [
    {
      "uri": str,
      "name": str,
      "description": str | None,
      "mimeType": str | None,
      "server": str,
    }
  ],
  "total": int,
}
```

ReadMcpResource

Tool name: ReadMcpResourceTool

Input:

```
{  
  "server": str, # The MCP server name  
  "uri": str, # The resource URI to read  
}
```

Output:

```
{  
  "contents": [  
    {"uri": str, "mimeType": str | None, "text": str | None,  
"blob": str | None}  
  ],  
  "server": str,  
}
```

Advanced Features with ClaudeSDKClient

Building a Continuous Conversation Interface

```
from claude_agent_sdk import (
    ClaudeSDKClient,
    ClaudeAgentOptions,
    AssistantMessage,
    TextBlock,
)
import asyncio

class ConversationSession:
    """Maintains a single conversation session with Claude."""

    def __init__(self, options: ClaudeAgentOptions | None =
None):
        self.client = ClaudeSDKClient(options)
        self.turn_count = 0

    async def start(self):
        await self.client.connect()
        print("Starting conversation session. Claude will
remember context.")
        print(
            "Commands: 'exit' to quit, 'interrupt' to stop
current task, 'new' for new session"
        )

        while True:
            user_input = input(f"\n[Turn {self.turn_count + 1}]
You: ")

            if user_input.lower() == "exit":
                break
            elif user_input.lower() == "interrupt":
                await self.client.interrupt()
                print("Task interrupted!")
                continue
            elif user_input.lower() == "new":
                # Disconnect and reconnect for a fresh session
                await self.client.disconnect()
                await self.client.connect()
```

```

        self.turn_count = 0
        print("Started new conversation session (previous
context cleared)")
        continue

    # Send message - the session retains all previous
messages

    await self.client.query(user_input)
    self.turn_count += 1

    # Process response
    print(f"[Turn {self.turn_count}] Claude: ", end="")
    async for message in self.client.receive_response():
        if isinstance(message, AssistantMessage):
            for block in message.content:
                if isinstance(block, TextBlock):
                    print(block.text, end="")
    print() # New line after response

    await self.client.disconnect()
    print(f"Conversation ended after {self.turn_count}
turns.")

async def main():
    options = ClaudeAgentOptions(
        allowed_tools=["Read", "Write", "Bash"],
        permission_mode="acceptEdits"
    )
    session = ConversationSession(options)
    await session.start()

# Example conversation:
# Turn 1 - You: "Create a file called hello.py"
# Turn 1 - Claude: "I'll create a hello.py file for you..."
# Turn 2 - You: "What's in that file?"
# Turn 2 - Claude: "The hello.py file I just created contains..."
(remembers!)
# Turn 3 - You: "Add a main function to it"
# Turn 3 - Claude: "I'll add a main function to hello.py..."
(knows which file!)

asyncio.run(main())

```


Using Hooks for Behavior Modification

```
from claude_agent_sdk import (
    ClaudeSDKClient,
    ClaudeAgentOptions,
    HookMatcher,
    HookContext,
)
import asyncio
from typing import Any

async def pre_tool_logger(
    input_data: dict[str, Any], tool_use_id: str | None, context:
HookContext
) -> dict[str, Any]:
    """Log all tool usage before execution."""
    tool_name = input_data.get("tool_name", "unknown")
    print(f"[PRE-TOOL] About to use: {tool_name}")

    # You can modify or block the tool execution here
    if tool_name == "Bash" and "rm -rf" in
str(input_data.get("tool_input", {})):
        return {
            "hookSpecificOutput": {
                "hookEventName": "PreToolUse",
                "permissionDecision": "deny",
                "permissionDecisionReason": "Dangerous command
blocked",
            }
        }
    return {}

async def post_tool_logger(
    input_data: dict[str, Any], tool_use_id: str | None, context:
HookContext
) -> dict[str, Any]:
    """Log results after tool execution."""
    tool_name = input_data.get("tool_name", "unknown")
    print(f"[POST-TOOL] Completed: {tool_name}")
    return {}
```

```

async def user_prompt_modifier(
    input_data: dict[str, Any], tool_use_id: str | None, context:
HookContext
) -> dict[str, Any]:
    """Add context to user prompts."""
    original_prompt = input_data.get("prompt", "")

    # Add a timestamp as additional context for Claude to see
    from datetime import datetime

    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    return {
        "hookSpecificOutput": {
            "hookEventName": "UserPromptSubmit",
            "additionalContext": f"[Submitted at {timestamp}]
Original prompt: {original_prompt}",
        }
    }

async def main():
    options = ClaudeAgentOptions(
        hooks={
            "PreToolUse": [
                HookMatcher(hooks=[pre_tool_logger]),
                HookMatcher(matcher="Bash", hooks=
[pre_tool_logger]),
            ],
            "PostToolUse": [HookMatcher(hooks=
[post_tool_logger])],
            "UserPromptSubmit": [HookMatcher(hooks=
[user_prompt_modifier])],
        },
        allowed_tools=["Read", "Write", "Bash"],
    )

    async with ClaudeSDKClient(options=options) as client:
        await client.query("List files in current directory")

    async for message in client.receive_response():
        # Hooks will automatically log tool usage
        pass

```

```
asyncio.run(main())
```

Real-time Progress Monitoring

```
from claude_agent_sdk import (
    ClaudeSDKClient,
    ClaudeAgentOptions,
    AssistantMessage,
    ToolUseBlock,
    ToolResultBlock,
    TextBlock,
)
import asyncio

async def monitor_progress():
    options = ClaudeAgentOptions(
        allowed_tools=["Write", "Bash"],
        permission_mode="acceptEdits"
    )

    async with ClaudeSDKClient(options=options) as client:
        await client.query("Create 5 Python files with different
sorting algorithms")

    # Monitor progress in real-time
    async for message in client.receive_response():
        if isinstance(message, AssistantMessage):
            for block in message.content:
                if isinstance(block, ToolUseBlock):
                    if block.name == "Write":
                        file_path =
block.input.get("file_path", "")
                        print(f"Creating: {file_path}")
                elif isinstance(block, ToolResultBlock):
                    print("Completed tool execution")
                elif isinstance(block, TextBlock):
                    print(f"Claude says:
{block.text[:100]}...")

            print("Task completed!")

asyncio.run(monitor_progress())
```

Example Usage

Basic file operations (using query)

```
from claude_agent_sdk import query, ClaudeAgentOptions,
AssistantMessage, ToolUseBlock
import asyncio

async def create_project():
    options = ClaudeAgentOptions(
        allowed_tools=["Read", "Write", "Bash"],
        permission_mode="acceptEdits",
        cwd="/home/user/project",
    )

    async for message in query(
        prompt="Create a Python project structure with setup.py",
        options=options
    ):
        if isinstance(message, AssistantMessage):
            for block in message.content:
                if isinstance(block, ToolUseBlock):
                    print(f"Using tool: {block.name}")

asyncio.run(create_project())
```

Error handling

```
from claude_agent_sdk import query, CLINotFoundError,
ProcessError, CLIJSONDecodeError

try:
    async for message in query(prompt="Hello"):
        print(message)
except CLINotFoundError:
    print(
        "Claude Code CLI not found. Try reinstalling: pip install
--force-reinstall claude-agent-sdk"
    )
except ProcessError as e:
    print(f"Process failed with exit code: {e.exit_code}")
except CLIJSONDecodeError as e:
    print(f"Failed to parse response: {e}")
```

Streaming mode with client

```
from claude_agent_sdk import ClaudeSDKClient
import asyncio

async def interactive_session():
    async with ClaudeSDKClient() as client:
        # Send initial message
        await client.query("What's the weather like?")

        # Process responses
        async for msg in client.receive_response():
            print(msg)

        # Send follow-up
        await client.query("Tell me more about that")

        # Process follow-up response
        async for msg in client.receive_response():
            print(msg)

asyncio.run(interactive_session())
```

Using custom tools with ClaudeSDKClient

```
from claude_agent_sdk import (
    ClaudeSDKClient,
    ClaudeAgentOptions,
    tool,
    create_sdk_mcp_server,
    AssistantMessage,
    TextBlock,
)

import asyncio
from typing import Any

# Define custom tools with @tool decorator
@tool("calculate", "Perform mathematical calculations",
{"expression": str})
async def calculate(args: dict[str, Any]) -> dict[str, Any]:
    try:
        result = eval(args["expression"], {"__builtins__": {}})
        return {"content": [{"type": "text", "text": f"Result:
{result}"}]}
    except Exception as e:
        return {
            "content": [{"type": "text", "text": f"Error:
{str(e)}"}],
            "is_error": True,
        }

@tool("get_time", "Get current time", {})
async def get_time(args: dict[str, Any]) -> dict[str, Any]:
    from datetime import datetime

    current_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    return {"content": [{"type": "text", "text": f"Current time:
{current_time}"}]}

async def main():
    # Create SDK MCP server with custom tools
    my_server = create_sdk_mcp_server(
        name="utilities", version="1.0.0", tools=[calculate,
get_time]
```



```

)

# Configure options with the server
options = ClaudeAgentOptions(
    mcp_servers={"utils": my_server},
    allowed_tools=["mcp__utils__calculate",
"mcp__utils__get_time"],
)

# Use ClaudeSDKClient for interactive tool usage
async with ClaudeSDKClient(options=options) as client:
    await client.query("What's 123 * 456?")

# Process calculation response
async for message in client.receive_response():
    if isinstance(message, AssistantMessage):
        for block in message.content:
            if isinstance(block, TextBlock):
                print(f"Calculation: {block.text}")

# Follow up with time query
await client.query("What time is it now?")

async for message in client.receive_response():
    if isinstance(message, AssistantMessage):
        for block in message.content:
            if isinstance(block, TextBlock):
                print(f"Time: {block.text}")

asyncio.run(main())

```

Sandbox Configuration

SandboxSettings

Configuration for sandbox behavior. Use this to enable command sandboxing and configure network restrictions programmatically.

```
class SandboxSettings(TypedDict, total=False):
    enabled: bool
    autoAllowBashIfSandboxed: bool
    excludedCommands: list[str]
    allowUnsandboxedCommands: bool
    network: SandboxNetworkConfig
    ignoreViolations: SandboxIgnoreViolations
    enableWeakerNestedSandbox: bool
```

Property	Type	Default	Description
enabled	bool	False	Enable sandbox mode for command execution
autoAllowBashIfSandboxed	bool	True	Auto-approve bash commands when sandbox is enabled
excludedCommands	list[str]	[]	Commands that always bypass sandbox restrictions (e.g., ["docker"]). These run unsandboxed automatically without model involvement
allowUnsandboxedCommands	bool	True	Allow the model to request running commands outside the sandbox. When <code>True</code> , the model can set <code>dangerouslyDisableSandbox</code> in tool input, which falls back to the <u>permissions system</u>
network	<u>SandboxNetworkConfig</u>	None	Network-specific sandbox configuration
ignoreViolations	<u>SandboxIgnoreViolations</u>	None	Configure which sandbox violations to ignore
enableWeakerNestedSandbox	bool	False	Enable a weaker nested sandbox for compatibility

❗ The sandbox depends on platform support and, on Linux, tools like `bubblewrap` and `socat`. By default, when `enabled` is `True` but the sandbox can't start, commands run unsandboxed with a warning on stderr. This default differs from the TypeScript SDK, where `failIfUnavailable` defaults to `true`.

Set `"failIfUnavailable": True` in your sandbox settings to stop instead. The key isn't declared on `SandboxSettings` yet, but the SDK forwards it to Claude Code, which honors it. `query()` then reports a `ResultMessage` with `subtype="error_during_execution"` and the reason in `errors`. Watch for that subtype rather than expecting `query()` to raise before yielding messages.

Example usage

```
from claude_agent_sdk import query, ClaudeAgentOptions,
SandboxSettings

sandbox_settings: SandboxSettings = {
    "enabled": True,
    "autoAllowBashIfSandboxed": True,
    "network": {"allowLocalBinding": True},
}

async for message in query(
    prompt="Build and test my project",
    options=ClaudeAgentOptions(sandbox=sandbox_settings),
):
    print(message)
```

 **Unix socket security:** The `allowUnixSockets` option can grant access to powerful system services. For example, allowing `/var/run/docker.sock` effectively grants full host system access through the Docker API, bypassing sandbox isolation. Only allow Unix sockets that are strictly necessary and understand the security implications of each.

SandboxNetworkConfig

Network-specific configuration for sandbox mode. These settings apply to sandboxed Bash commands when `enabled` is `True` in the parent `SandboxSettings`. They do not restrict the WebFetch tool, which uses permission rules instead.

```
class SandboxNetworkConfig(TypedDict, total=False):
    allowedDomains: list[str]
    deniedDomains: list[str]
    allowManagedDomainsOnly: bool
    allowUnixSockets: list[str]
    allowAllUnixSockets: bool
    allowLocalBinding: bool
    allowMachLookup: list[str]
    httpProxyPort: int
    socksProxyPort: int
```

Property	Type	Default	Description
allowedDomains	list[str]	[]	Domain names that sandboxed processes can access
deniedDomains	list[str]	[]	Domain names that sandboxed processes cannot access. Takes precedence over allowedDomains
allowManagedDomains Only	bool	False	Managed-settings only: when set in managed settings, ignore allowedDomains from non-managed settings sources. Has no effect when set via SDK options
allowUnixSockets	list[str]	[]	Unix socket paths that processes can access (e.g., Docker socket)
allowAllUnixSockets	bool	False	Allow access to all Unix sockets
allowLocalBinding	bool	False	Allow processes to bind to local ports (e.g., for dev servers)
allowMachLookup	list[str]	[]	macOS only: XPC/Mach service names to allow. Supports a trailing wildcard
httpProxyPort	int	None	HTTP proxy port for network requests
socksProxyPort	int	None	SOCKS proxy port for network requests

ⓘ The built-in sandbox proxy enforces the network allowlist based on the requested hostname and does not terminate or inspect TLS traffic, so techniques such as **domain fronting** can potentially bypass it. See [Sandboxing security limitations](#) for details and [Secure deployment](#) for configuring a TLS-terminating proxy.

SandboxIgnoreViolations

Configuration for ignoring specific sandbox violations.

```
class SandboxIgnoreViolations(TypedDict, total=False):
    file: list[str]
    network: list[str]
```

Property	Type	Default	Description
file	list[str]	[]	File path patterns to ignore violations for

Property	Type	Default	Description
network	list[str]	[]	Network patterns to ignore violations for

Permissions Fallback for Unsandboxed Commands

When `allowUnsandboxedCommands` is enabled, the model can request to run commands outside the sandbox by setting `dangerouslyDisableSandbox: True` in the tool input. These requests fall back to the existing permissions system, meaning your `can_use_tool` handler will be invoked, allowing you to implement custom authorization logic.



`excludedCommands` vs `allowUnsandboxedCommands` :

- `excludedCommands` : A static list of commands that always bypass the sandbox automatically (e.g., `["docker"]`). The model has no control over this.
- `allowUnsandboxedCommands` : Lets the model decide at runtime whether to request unsandboxed execution by setting `dangerouslyDisableSandbox: True` in the tool input.

```

from claude_agent_sdk import (
    query,
    ClaudeAgentOptions,
    HookMatcher,
    PermissionResultAllow,
    PermissionResultDeny,
    ToolPermissionContext,
)

async def can_use_tool(
    tool: str, input: dict, context: ToolPermissionContext
) -> PermissionResultAllow | PermissionResultDeny:
    # Check if the model is requesting to bypass the sandbox
    if tool == "Bash" and input.get("dangerouslyDisableSandbox"):
        # The model is requesting to run this command outside the
        sandbox
        print(f"Unsandboxed command requested:
{input.get('command')}")

        if is_command_authorized(input.get("command")):
            return PermissionResultAllow()
        return PermissionResultDeny(
            message="Command not authorized for unsandboxed
execution"
        )
    return PermissionResultAllow()

# Required: dummy hook keeps the stream open for can_use_tool
async def dummy_hook(input_data, tool_use_id, context):
    return {"continue_": True}

async def prompt_stream():
    yield {
        "type": "user",
        "message": {"role": "user", "content": "Deploy my
application"},
    }

async def main():
    async for message in query(

```

```

prompt=prompt_stream(),
options=ClaudeAgentOptions(
    sandbox={
        "enabled": True,
        "allowUnsandboxedCommands": True, # Model can
request unsandboxed execution
    },
    permission_mode="default",
    can_use_tool=can_use_tool,
    hooks={"PreToolUse": [HookMatcher(matcher=None,
hooks=[dummy_hook])]},
),
):
    print(message)

```

This pattern enables you to:

- **Audit model requests:** Log when the model requests unsandboxed execution
- **Implement allowlists:** Only permit specific commands to run unsandboxed
- **Add approval workflows:** Require explicit authorization for privileged operations

⚠ Commands running with `dangerouslyDisableSandbox: True` have full system access. Ensure your `can_use_tool` handler validates these requests carefully.

If `permission_mode` is set to `bypassPermissions` and `allow_unsandboxed_commands` is enabled, the model can autonomously execute commands outside the sandbox without approval prompts (an explicit ask rule still forces one). This combination effectively allows the model to escape sandbox isolation silently.

See also

- [SDK overview](#) - General SDK concepts
- [TypeScript SDK reference](#) - TypeScript SDK documentation
- [CLI reference](#) - Command-line interface
- [Common workflows](#) - Step-by-step guides